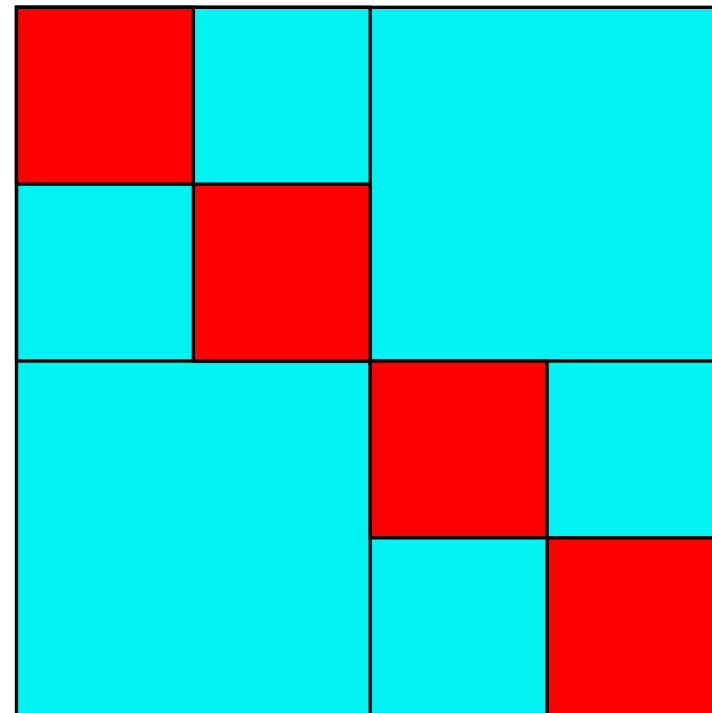
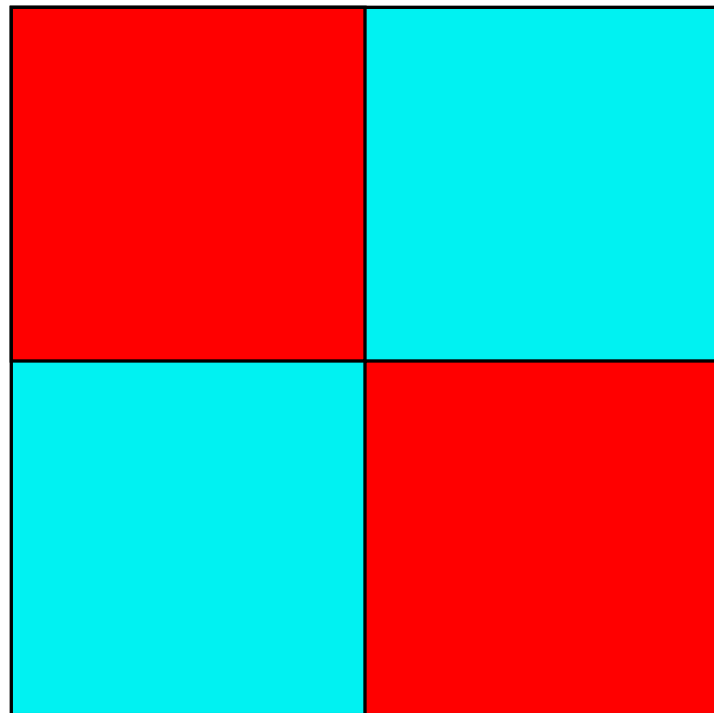


Fast direct solvers for hierarchical matrices

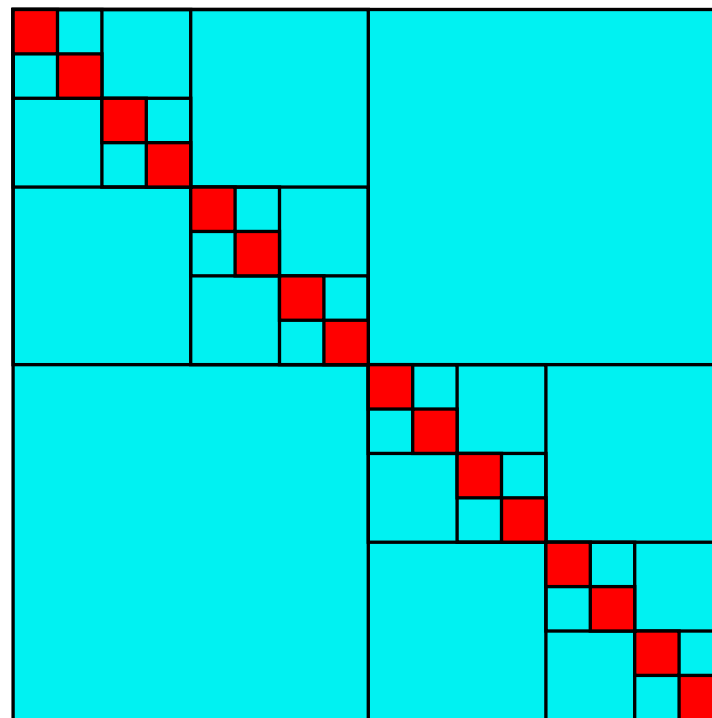
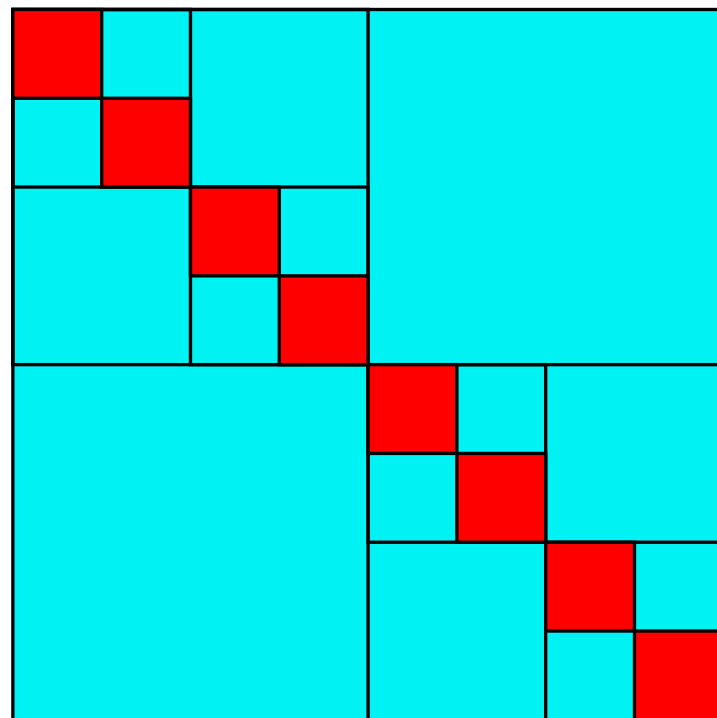
Day 5

What is a hierarchical matrix?

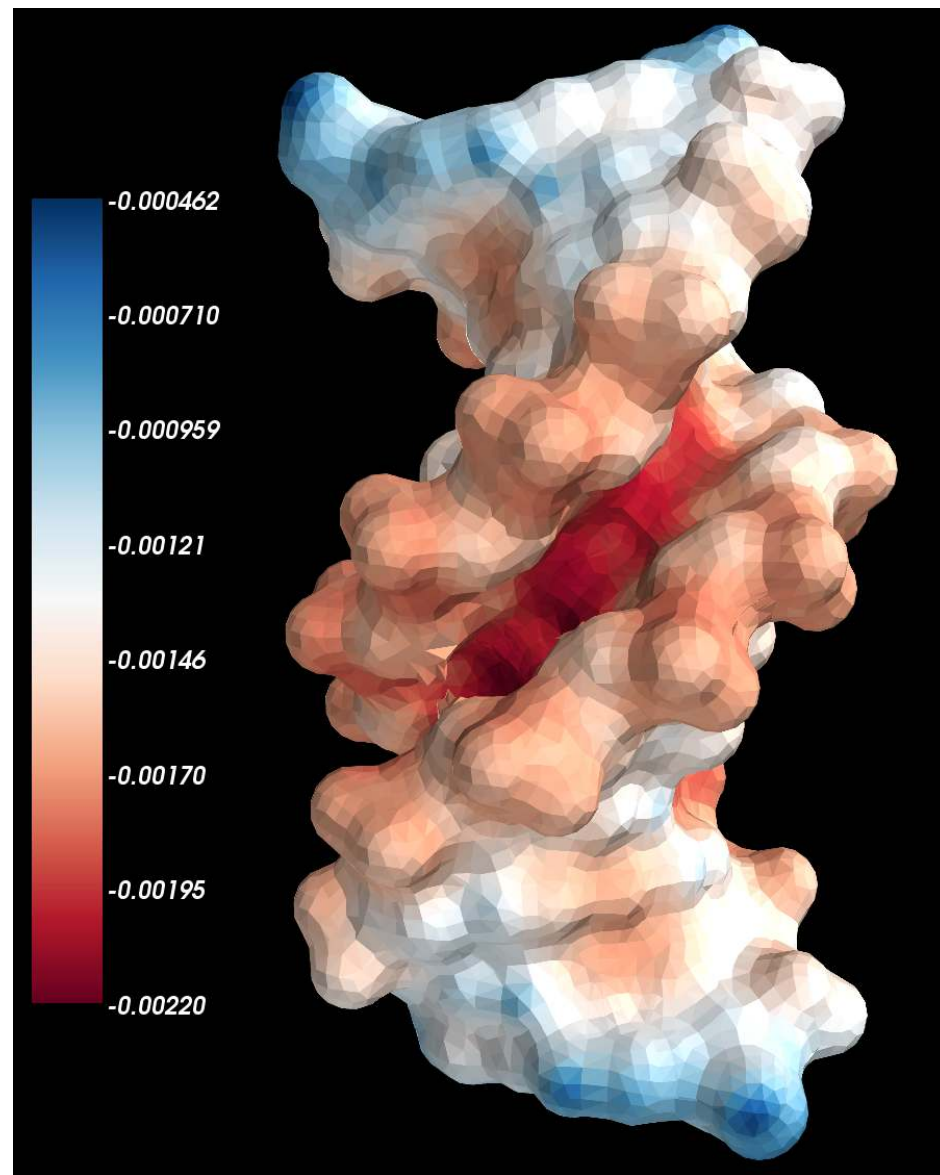


■ → Full rank

■ → Low rank



Applications - Molecular dynamics



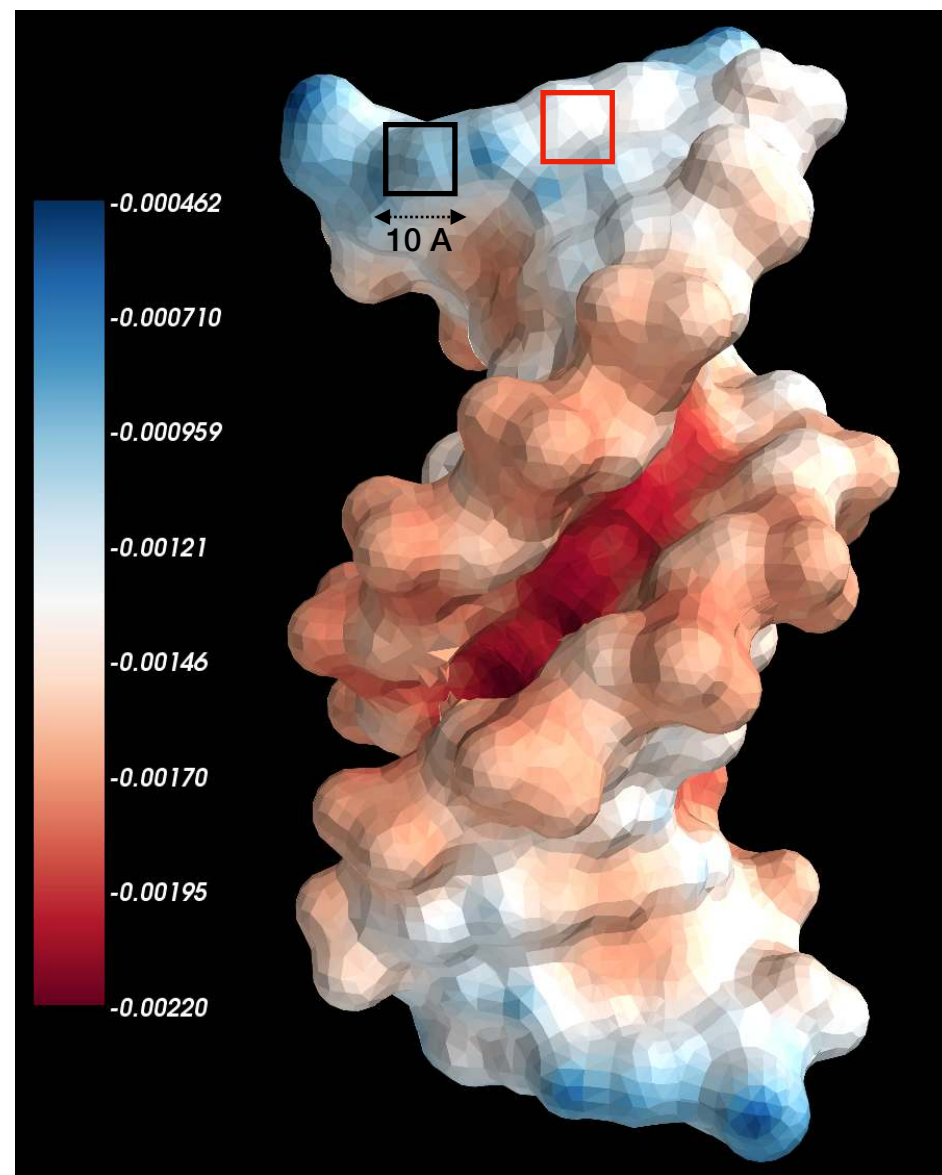
- pKa computation
- Docking

$$-\Delta\varphi = 0 \quad \text{in } \Omega_0$$

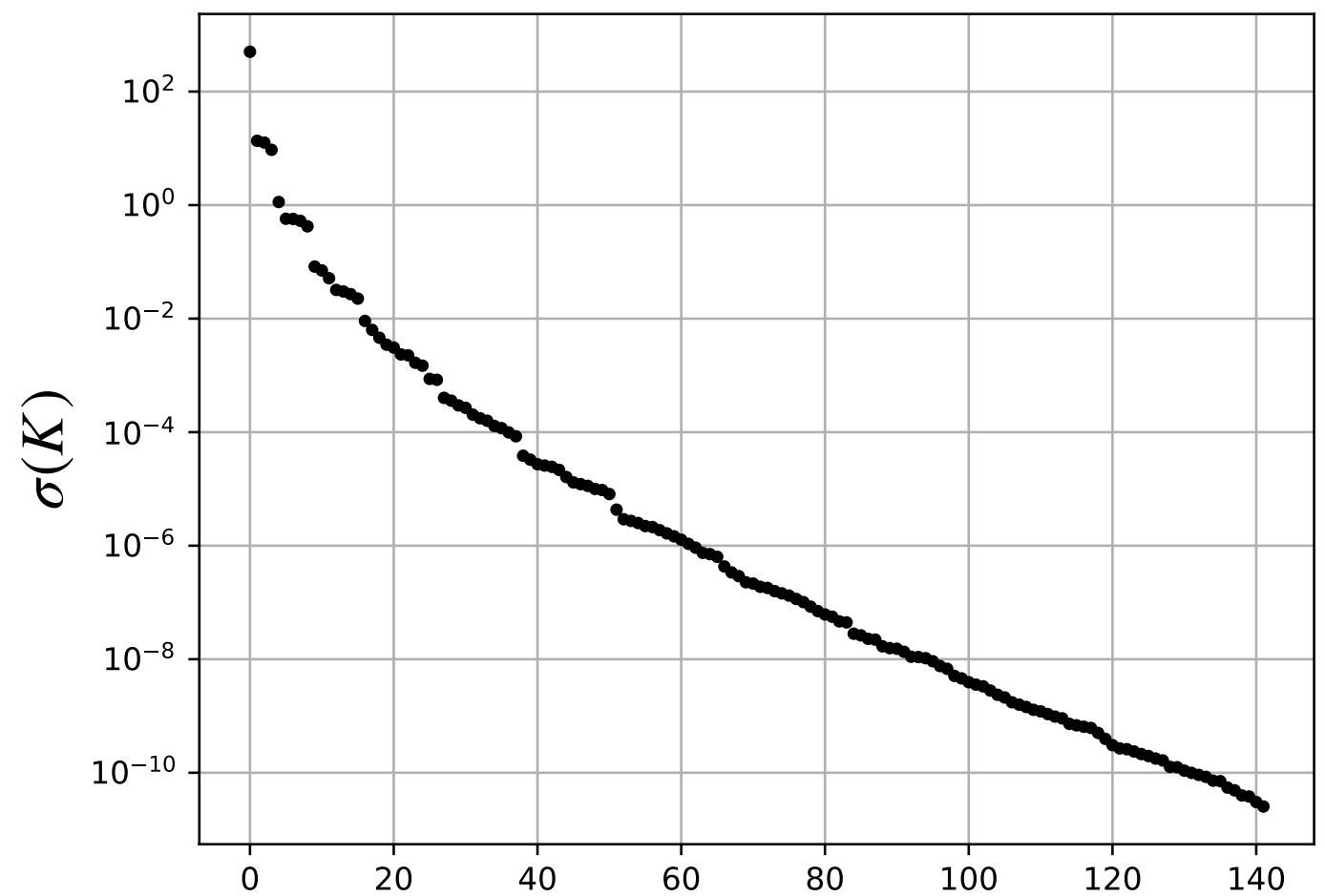
$$-\Delta\varphi = \frac{1}{\varepsilon_1} \sum_i q_i \delta(\mathbf{r} - \mathbf{r}_i) \quad \text{in } \Omega_1$$

$$[\varphi] = \left[\varepsilon \frac{\partial \varphi}{\partial \nu} \right] = 0 \quad \text{on } \Sigma$$

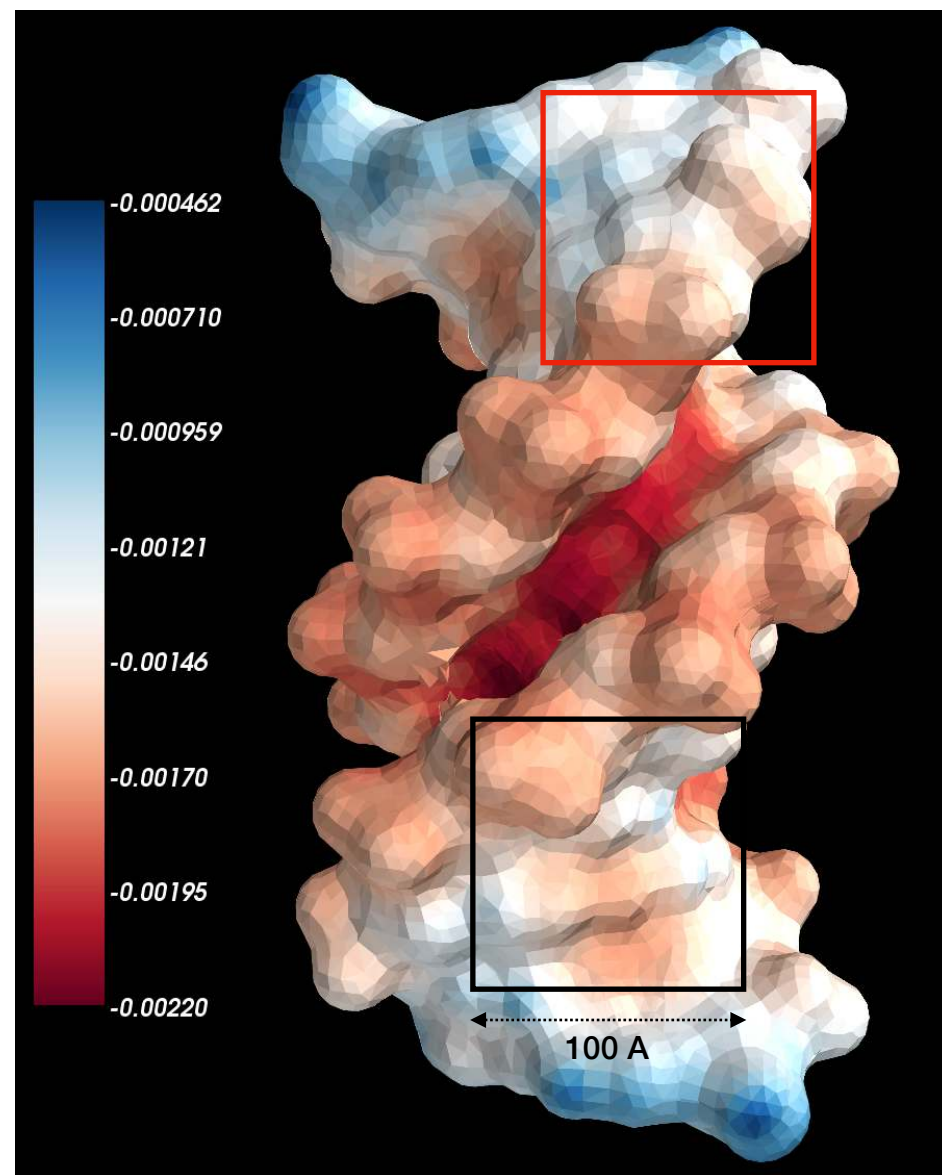
Applications - Molecular dynamics



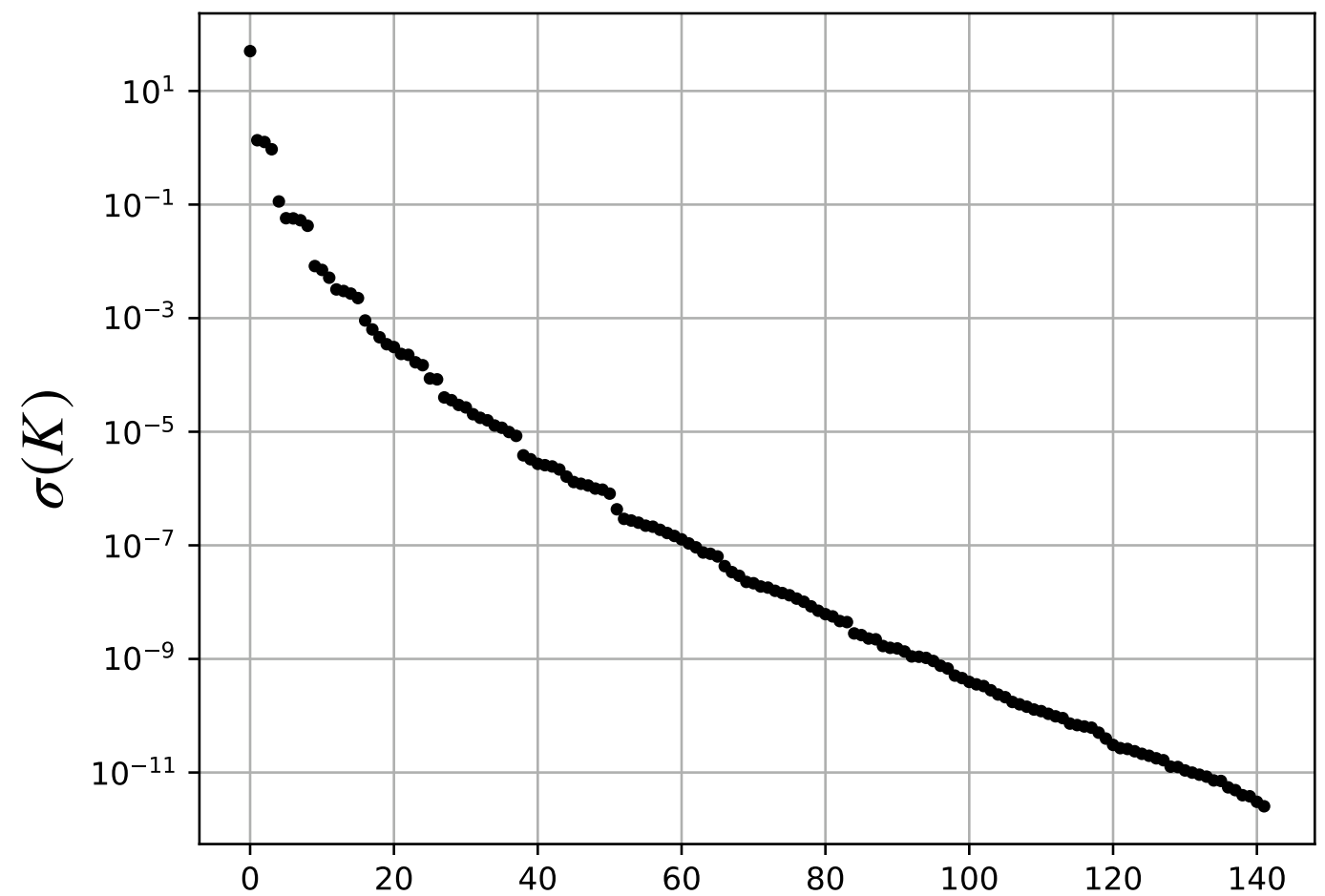
$$K_{i,j} = \frac{1}{|x_i - x_j|}$$



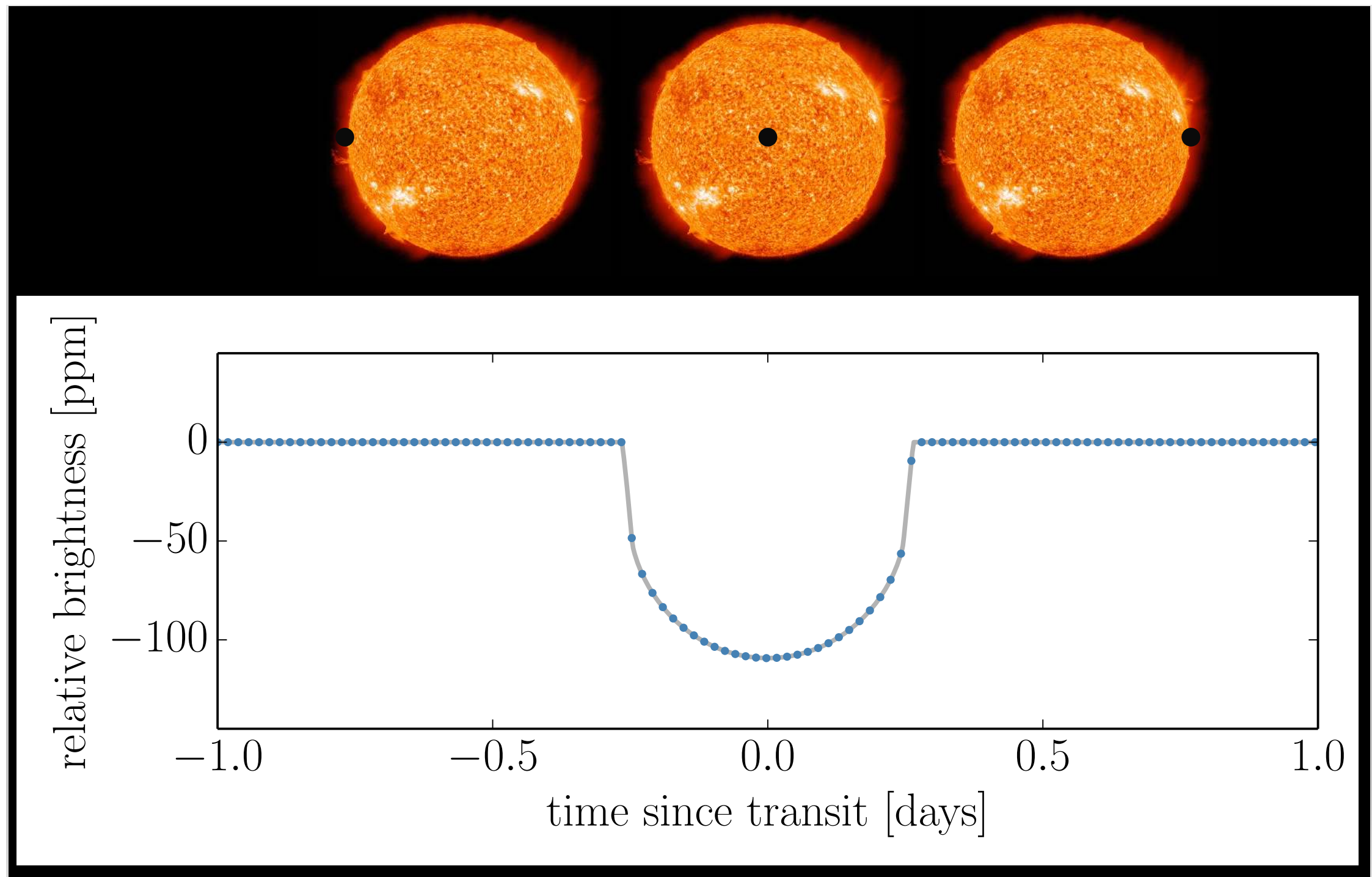
Applications - Molecular dynamics



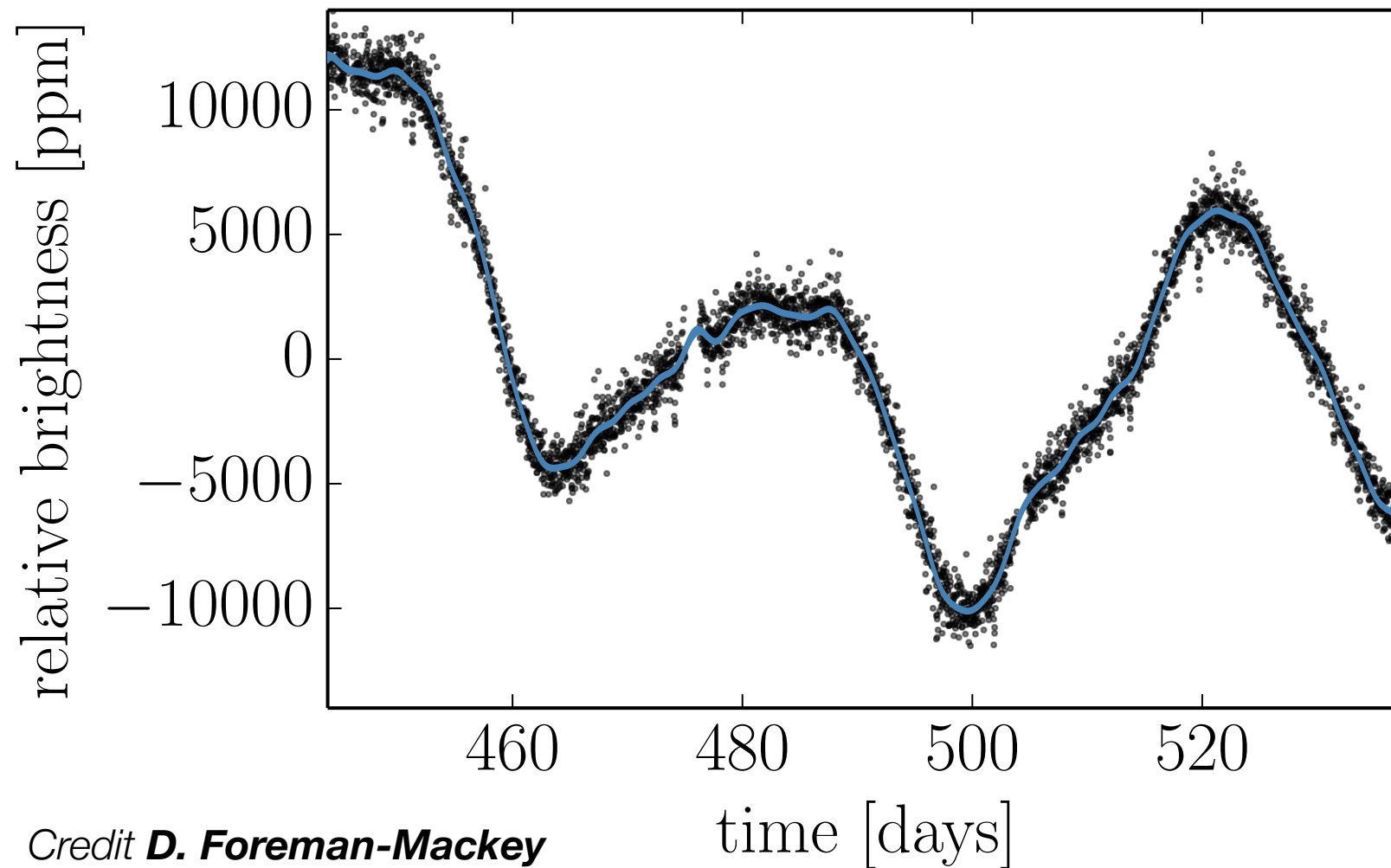
$$K_{i,j} = \frac{1}{|x_i - x_j|}$$



Exoplanet hunting using Gaussian Processes

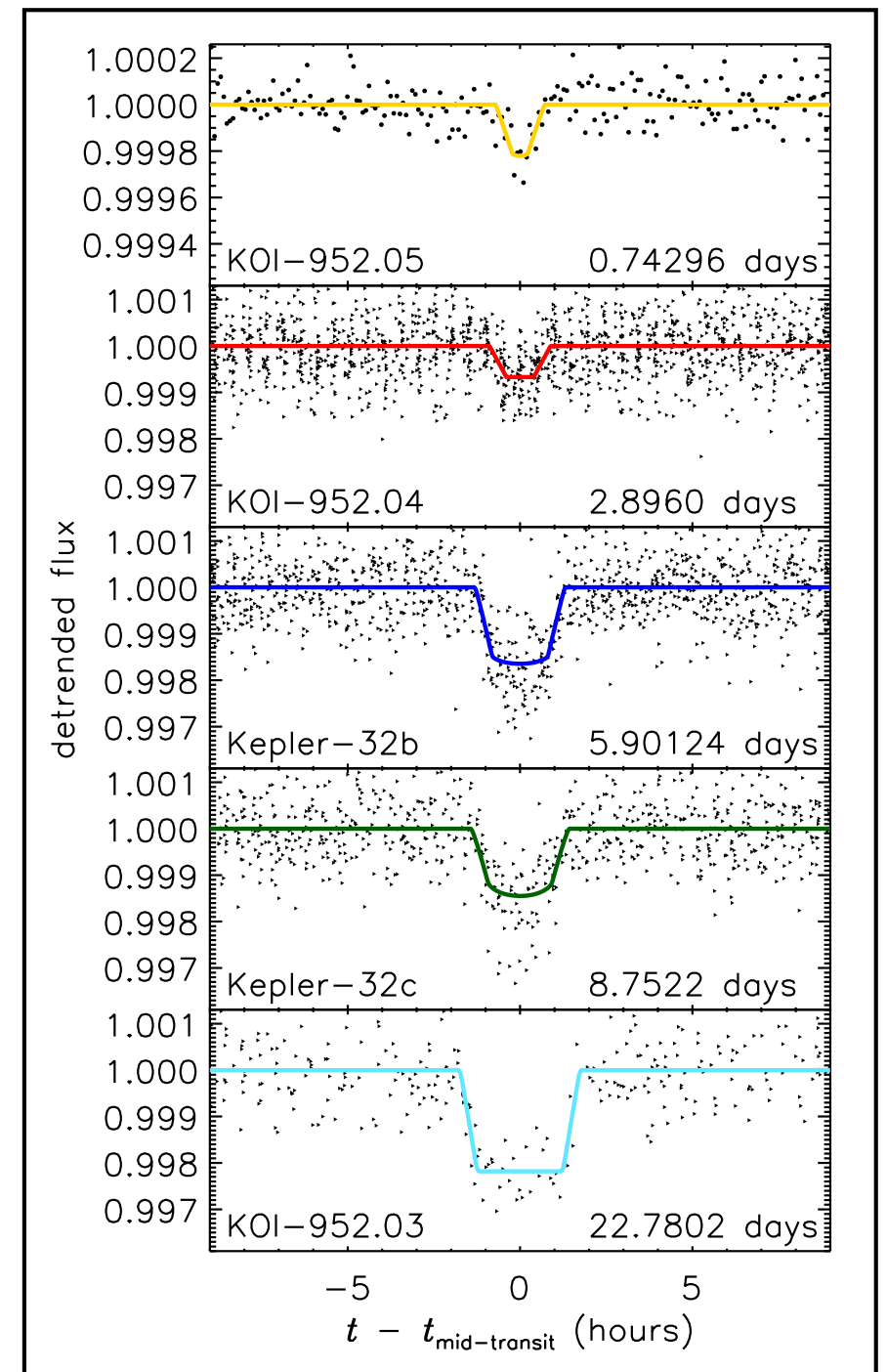


Exoplanet hunting using Gaussian Processes



Computational Task:

$$\operatorname{argmax}_{\theta} \mathcal{L}_{\theta} \propto \frac{1}{\det C(t; \theta)^{1/2}} e^{-\frac{1}{2} \mathbf{y}^T C^{-1}(t; \theta) \mathbf{y}}$$



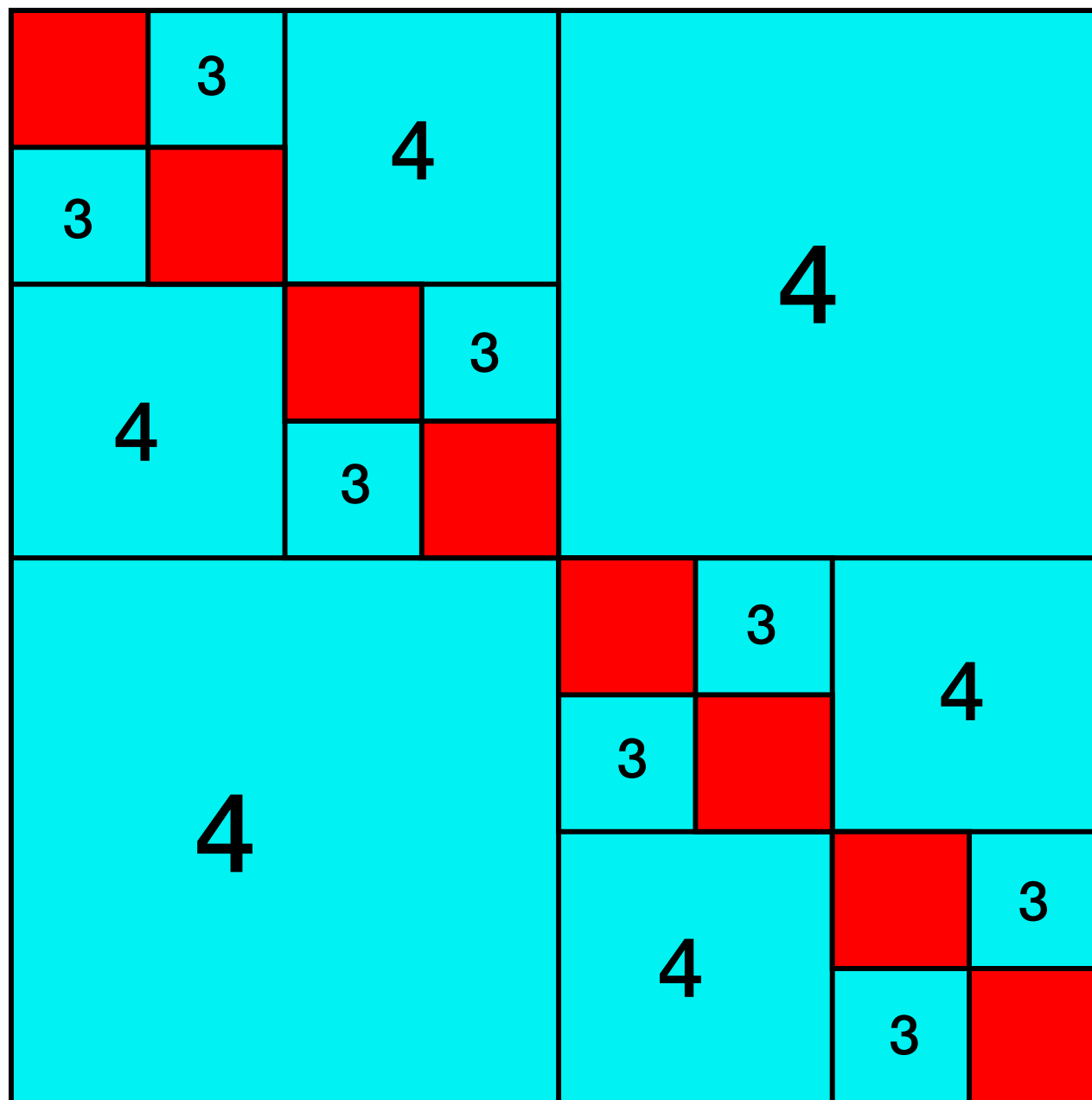
Credit **Fabrycky et al. (2012)**

Exoplanet hunting using Gaussian Processes

Computational Task:

$$\operatorname{argmax}_{\theta} \mathcal{L}_{\theta} \propto \frac{1}{\det C(t; \theta)^{1/2}} e^{-\frac{1}{2} y^T C^{-1}(t; \theta) y}$$

$$C(t, \theta) = \sigma_{\varepsilon}^2 I + K(t, t'; \theta)$$



$$K(t, t', \theta) = \theta(0) + e^{-\frac{(t-t')^2}{2\theta(1)^2}}$$

$$\theta(0) = 1, \theta(1) = 3$$



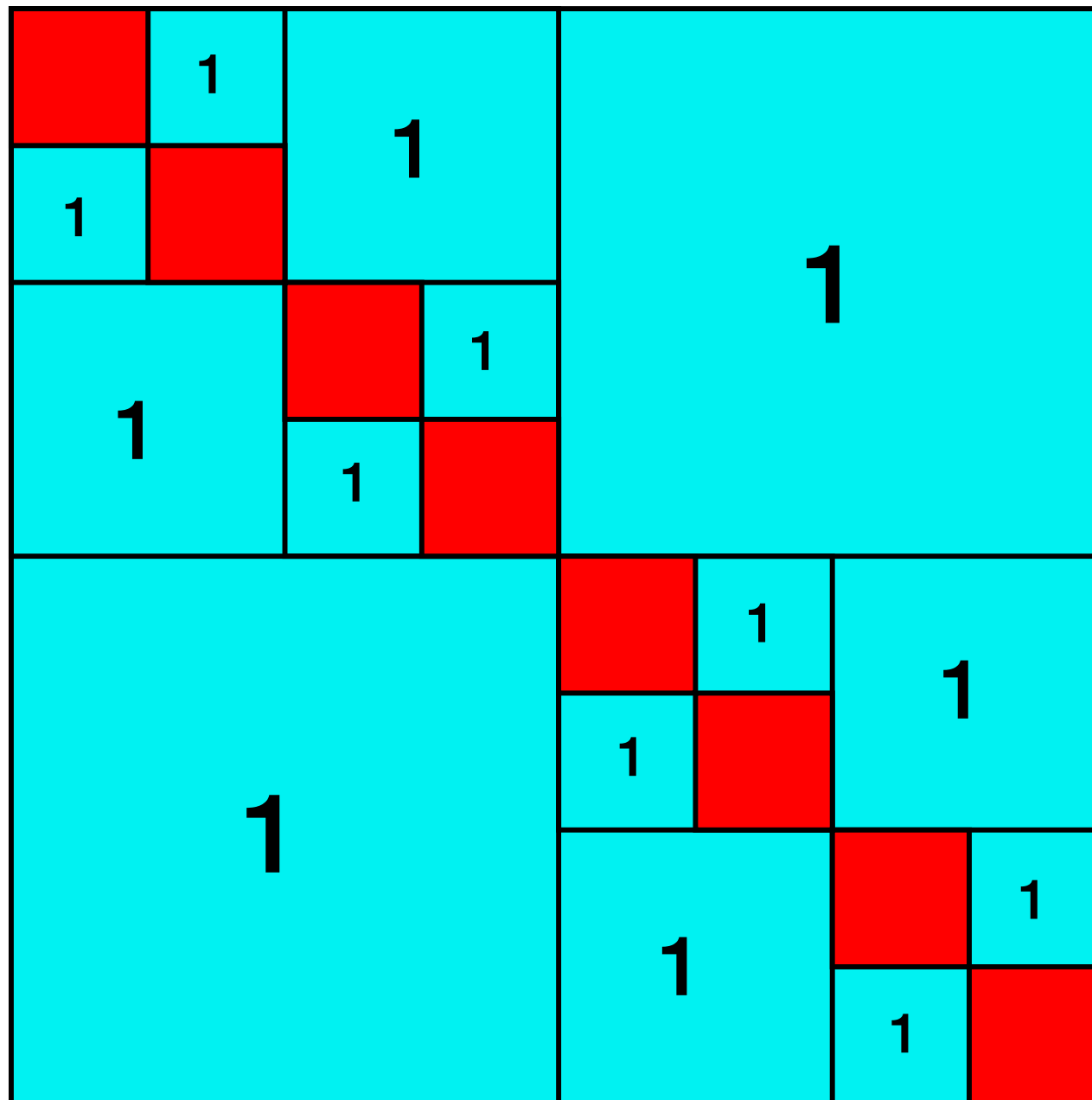
10k t'_i s uniformly distributed on $[0, 1]$

Exoplanet hunting using Gaussian Processes

Computational Task:

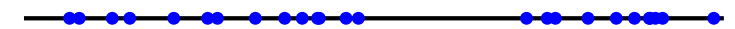
$$\operatorname{argmax}_{\theta} \mathcal{L}_{\theta} \propto \frac{1}{\det C(t; \theta)^{1/2}} e^{-\frac{1}{2} y^T C^{-1}(t; \theta) y}$$

$$C(t, \theta) = \sigma_{\varepsilon}^2 I + K(t, t'; \theta)$$



$$K(t, t', \theta) = \theta(0) + e^{-\theta(1)|t-t'|}$$

$$\theta(0) = 1, \theta(1) = 3$$



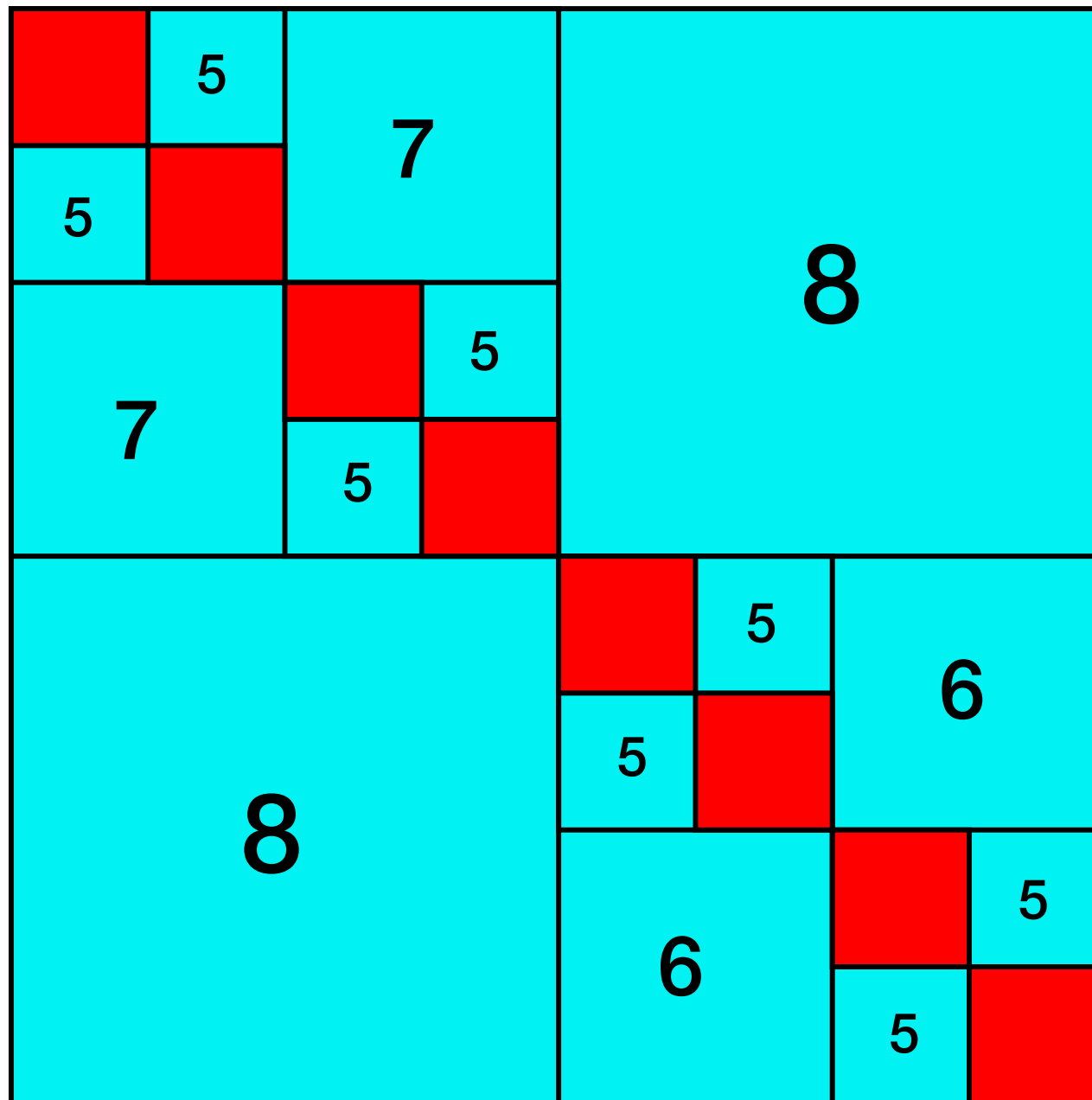
10k t'_i 's uniformly distributed on $[0,1]$

Exoplanet hunting using Gaussian Processes

Computational Task:

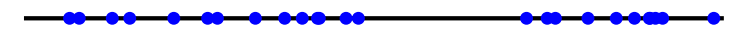
$$\operatorname{argmax}_{\theta} \mathcal{L}_{\theta} \propto \frac{1}{\det C(t; \theta)^{1/2}} e^{-\frac{1}{2} y^T C^{-1}(t; \theta) y}$$

$$C(t, \theta) = \sigma_{\varepsilon}^2 I + K(t, t'; \theta)$$



$$K(t, t', \theta) = \theta(0) + \frac{1}{1 + (x(i) - x(j))^2}$$

$$\theta(0) = 1$$



10k t'_i s uniformly distributed on $[0,1]$

What is fast?

Suppose $A \in \mathbb{R}^{n \times n}$, and $v \in \mathbb{R}^n$

- Matrix vector product (matvec) $A \cdot v$: $O(n^2)$
- Inversion A^{-1} : $O(n^3)$
- Determinants $\det A$: $O(n^3)$

For a given task, an algorithm is fast if it's runtime beats the asymptotic complexity

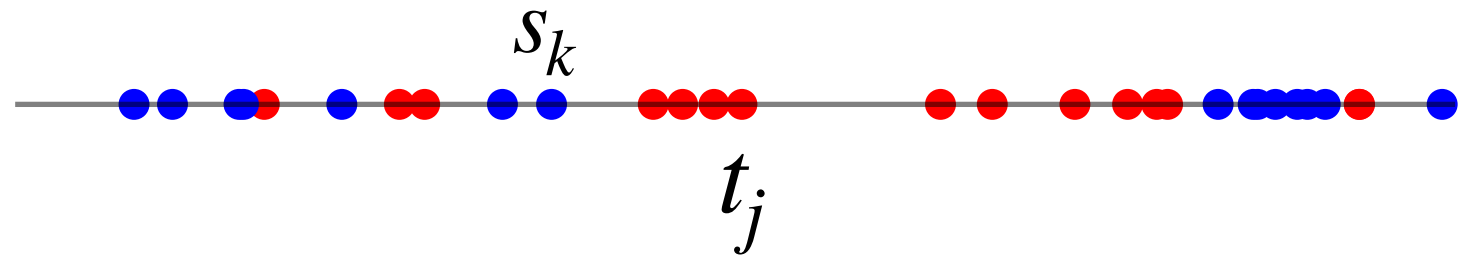
The dream: $O(n \log^s n)$

Examples

- Sparse matrices, matvecs in $O(kn)$, if well-conditioned, inverse in $O(kn)$
- FFT matrices, matvecs in $O(n \log n)$, inverse analytically known, and inverse application in $O(n \log n)$

Dense matrices $\neq$ Data dense

$$\begin{aligned} A_{j,k} &= \delta_{j,k} + \cos(t_j - s_k) \\ &= \delta_{j,k} + \cos(t_j)\cos(s_k) + \sin(t_j)\sin(s_k) \end{aligned}$$



- **Matvec** $b = A \cdot v : O(n^2)$

Step 1:

$$W_1 = \sum_{k=1}^n \cos(s_k) v_k, \quad W_2 = \sum_{k=1}^n \sin(s_k) v_k$$

Step 2:

$$b_j = v_j + \cos(t_j) W_1 + \sin(t_j) W_2 \quad O(n)!$$

$$A = I + UV^T, \quad U = \begin{bmatrix} \cos(t_1) & \sin(t_1) \\ \cos(t_2) & \sin(t_2) \\ \vdots & \vdots \\ \cos(t_n) & \sin(t_n) \end{bmatrix}, \quad V = \begin{bmatrix} \cos(s_1) & \sin(s_1) \\ \cos(s_2) & \sin(s_2) \\ \vdots & \vdots \\ \cos(s_n) & \sin(s_n) \end{bmatrix}$$

- **Inversion** $A^{-1} : O(n^3)$

Sherman Morrison Woodbury formula: $A^{-1} = I - \underset{n \times 2}{U} (\underset{2 \times 2}{I_2} + \underset{2 \times n}{V^T U})^{-1} \underset{2 \times n}{V^T}$ $O(n)!$

- **Determinants** $\det A : O(n^3)$

Sylvester formula formula: $\det A = \det(I_2 + V^T U)$ $O(n)!$

Low-rank approximations - Functional SVDs

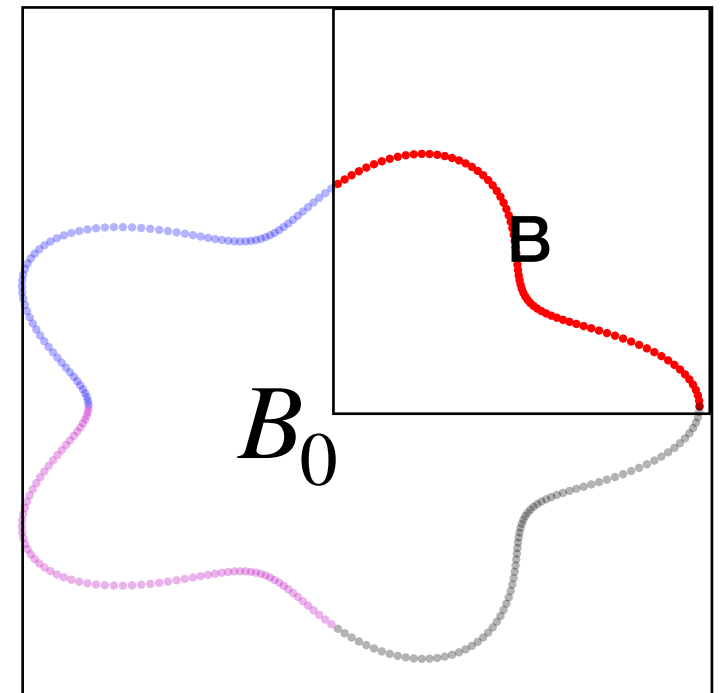
$A_{ij} = K(x_i, x_j)$, where $x_i \in \mathbb{R}^d$

$K(x, y)$ is a smooth function except when $x = y$

$$Tf = \int_B K(x, y)f(y)dy, \quad T: \mathbb{L}^2(B_0 \setminus B) \rightarrow \mathbb{L}^2(B) \text{ with}$$
$$\int_{B_0 \setminus B} \int_B |K(x, y)|^2 dx dy < \infty$$

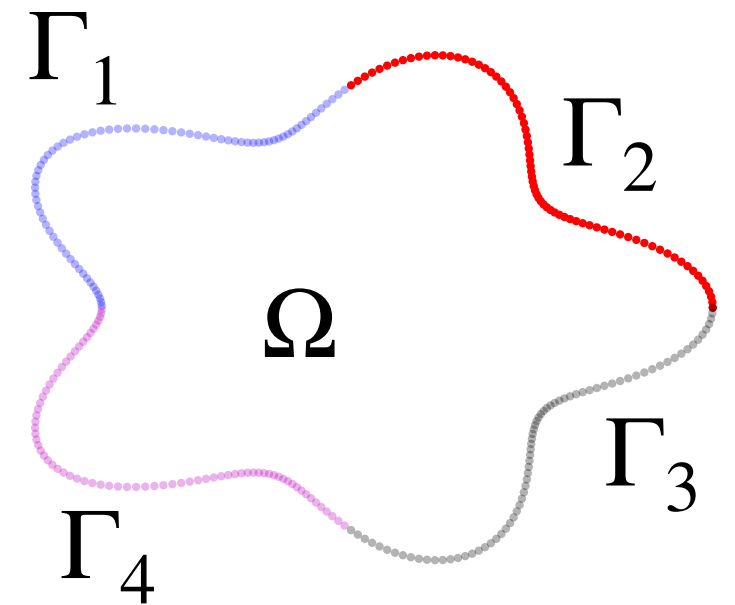
- Then $K(x, y) = \sum_{i=1}^p u_i(x)s_i v_i(y) + O(\varepsilon)$

- Computing the functional SVD can be numerically intensive, particularly beyond $d=2,3$
- Costs can be amortized for translationally invariant kernels $K(x, y) = K(|x - y|)$ and/or homogeneous kernels $K(\lambda x, \lambda y) = \lambda^\alpha K(x, y)$
- Computational savings if kernel satisfies Green's identities (Proxy surfaces)



Low rank approximations - Randomized algorithms

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ \color{red}{A_{21}} & A_{22} & \color{red}{A_{23}} & \color{red}{A_{24}} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix} \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \\ \sigma_4 \end{bmatrix}$$



$$\begin{matrix} A_2 \\ [A_{21} & 0 & A_{23} & A_{24}] \\ p \times N & & k \times N & k \times p \end{matrix} = U_2 \begin{bmatrix} \tilde{V}_{21}^T & 0 & \tilde{V}_{23}^T & \tilde{V}_{24}^T \end{bmatrix}$$

Randomized algorithms:

$W = \mathbb{R}^{N \times (k+r)}$, **random Gaussian matrix, FFT matrix**

$Y = A_2 W$ $Y \in \mathbb{R}^{p \times (k+r)}$ Sample range of matrix

$Y = QR$ Orthogonalize sampled range

$T = Q^* A_2$ $T \in \mathbb{R}^{(k+r) \times p}$ Change of basis

$T = \hat{U} S V^T$ SVD of reduced matrix

$A_2 \approx Q \hat{U} S V^T$

Randomized algorithms - error analysis and performance

$$\begin{array}{c} A_2 \\ [A_{21} \quad 0 \quad A_{23} \quad A_{24}] = U_2 [\tilde{V}_{21}^T \quad 0 \quad \tilde{V}_{23}^T \quad \tilde{V}_{24}^T] \\ p \times N \qquad k \times N \qquad k \times p \end{array}$$

Randomized algorithms:

$W = \mathbb{R}^{N \times (k+r)}$, **random Gaussian matrix, FFT matrix**

$Y = A_2 W$ $Y \in \mathbb{R}^{p \times (k+r)}$ Sample range of matrix $O(N \cdot (k+r) \cdot p)$

$Y = QR$ Orthogonalize sampled range $O(p \cdot (k+r)^2)$

$T = Q^* A_2$ $T \in \mathbb{R}^{(k+r) \times N}$ Change of basis $O(N \cdot (k+r) \cdot p)$

$T = \hat{U} S V^T$ SVD of reduced matrix $O((k+r)^2 \cdot N)$

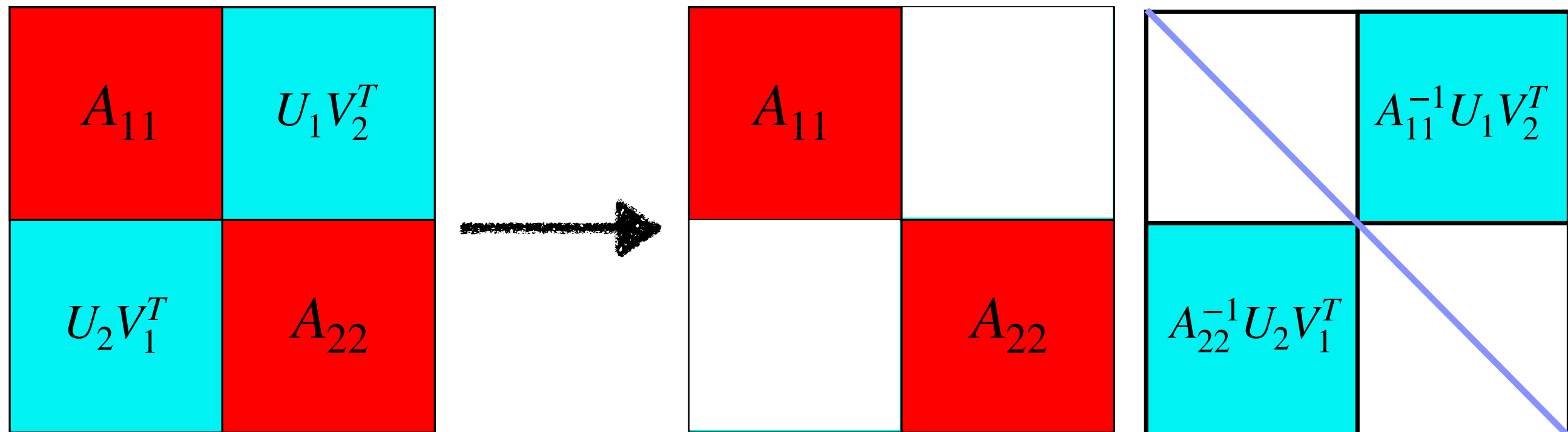
$$A_2 \approx Q \hat{U} S V^T$$

$$\|A_2 - Q \hat{U} S V^T\| = \|A_2 - Q T\| = \|A_2 - Q Q^* A_2\|$$

$$\|A_2 - Q Q^* A_2\| \leq \left(1 + C\sqrt{N}\right) s_{k+1} \quad \text{with probability } 1 - 6r^{-r}$$

HODLR: One level scheme - factorization

Assume: All off-diagonal blocks are rank r



Full rank;
 Low-rank;

 Identity matrix;
 Zero matrix;

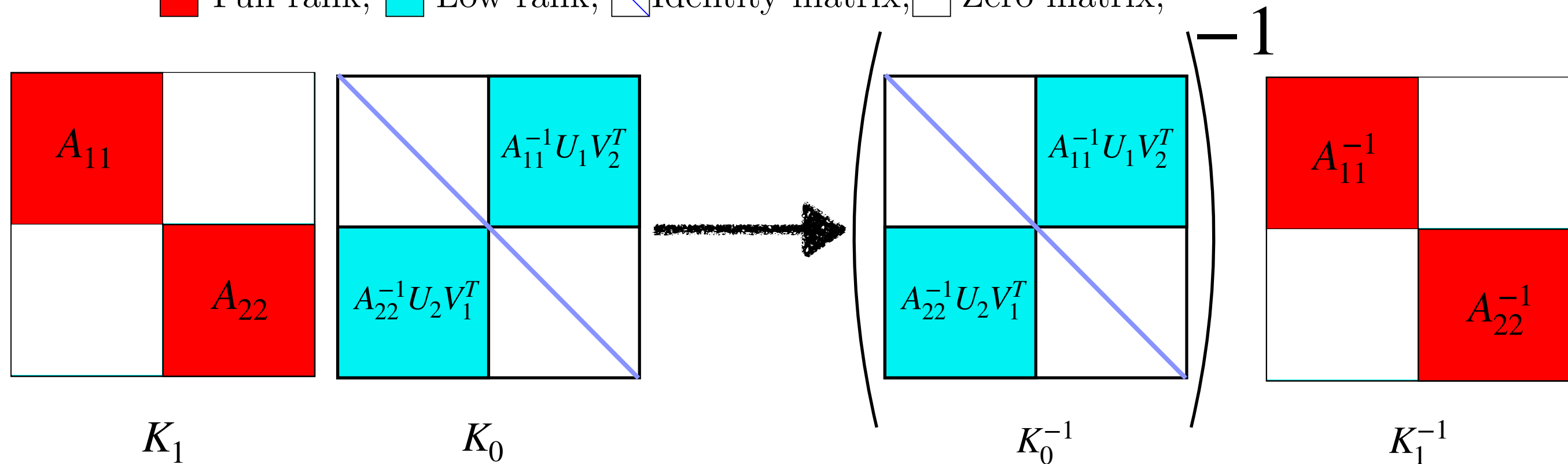
Factorization tasks and costs:

Compute $A_{12} = U_1 V_2^T$, and $A_{21} = U_2 V_1^T$: $2 \cdot O\left(\frac{n^2}{4} \cdot r\right)$ Factorization Cost: $\frac{n^3}{4}$

Compute A_{11}^{-1} , and A_{22}^{-1} : $2 \cdot \frac{n^3}{8}$

One level scheme - Inversion

■ Full rank;
 ■ Low-rank;
 ▬ Identity matrix;
 Zero matrix;



$$K_0 = I_n + \begin{bmatrix} & \tilde{U}_1 \\ \tilde{U}_2 & \end{bmatrix} \begin{bmatrix} & V_2^T \\ V_1^T & \end{bmatrix}$$

$\leftarrow \dots \dots \dots \rightarrow$
 n

$\updownarrow \dots \dots \dots \updownarrow$
 $2r$

K_0

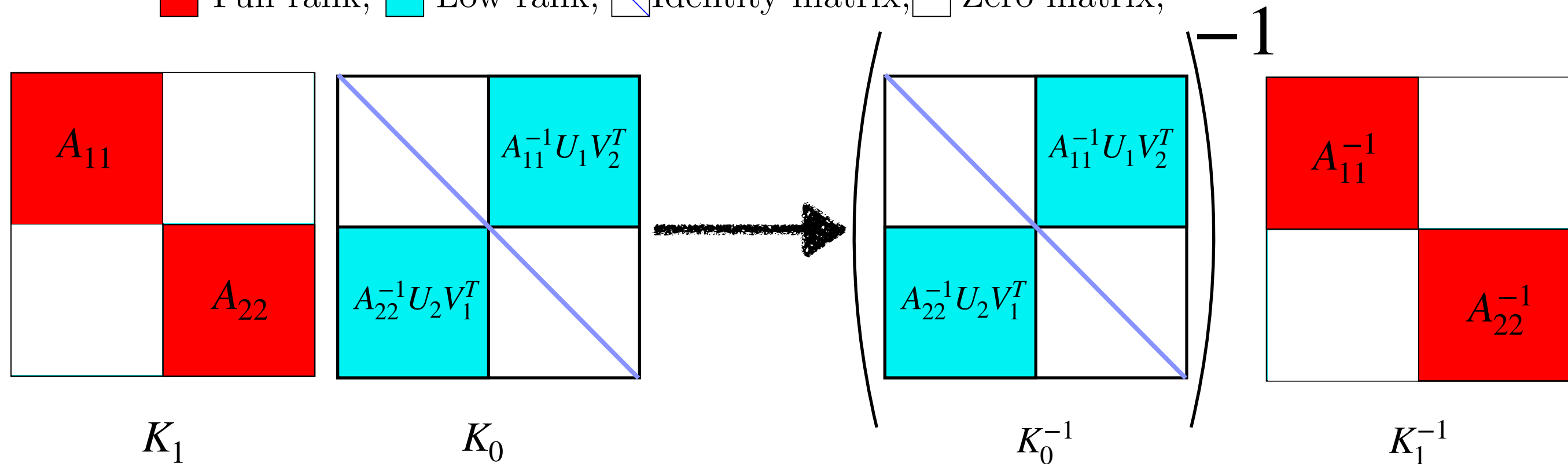
K_0^{-1} using Sherman Morrison
 Woodbury formula in $\mathcal{O}(4nr^2)$

$$I_n - U (I_{2r} + V^T U)^{-1} V^T$$

One level scheme - Inversion

Full rank;
 Low-rank;

 Identity matrix;
 Zero matrix;



$$K_0 = I_n + \begin{bmatrix} & \tilde{U}_1 \\ \tilde{U}_2 & \end{bmatrix} \begin{bmatrix} & V_2^T \\ V_1^T & \end{bmatrix}$$

$\leftarrow \dots \dots \dots \rightarrow$
 n

$\updownarrow \dots \dots \dots \updownarrow$
 $2r$

K_0

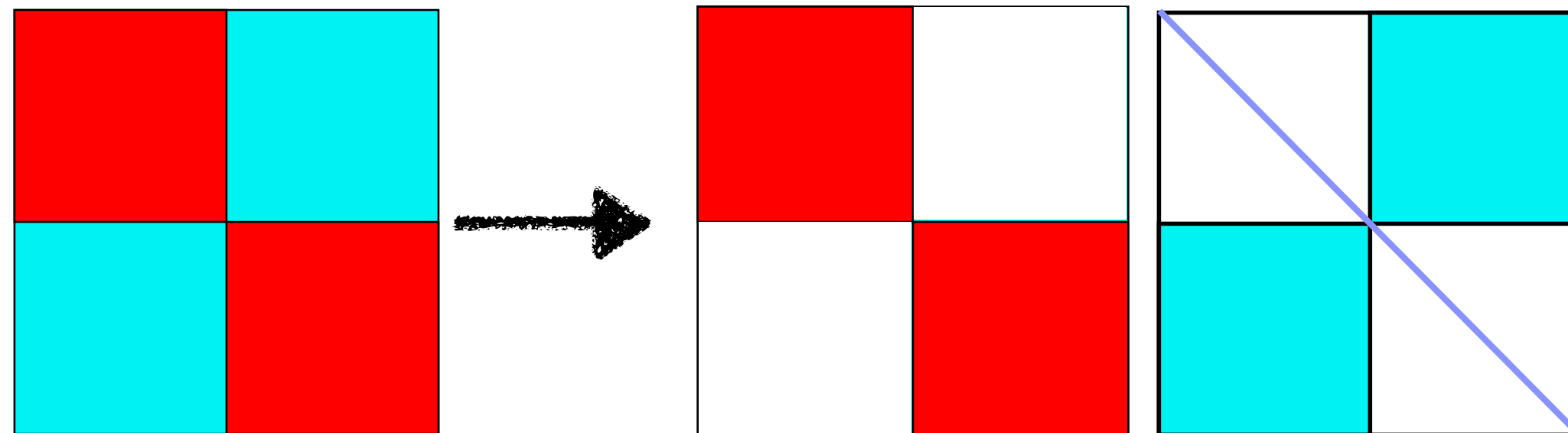
Inversion Cost:

$$n^3 \rightarrow \frac{n^2}{2}$$

Factorization Cost:

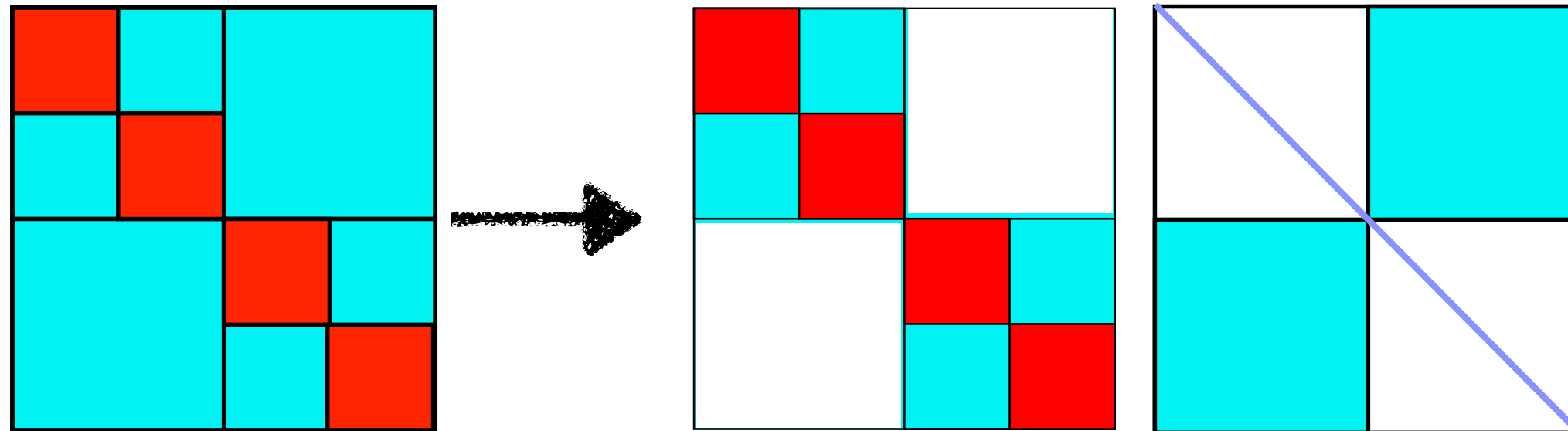
$$\frac{n^3}{4}$$

Can we induct?



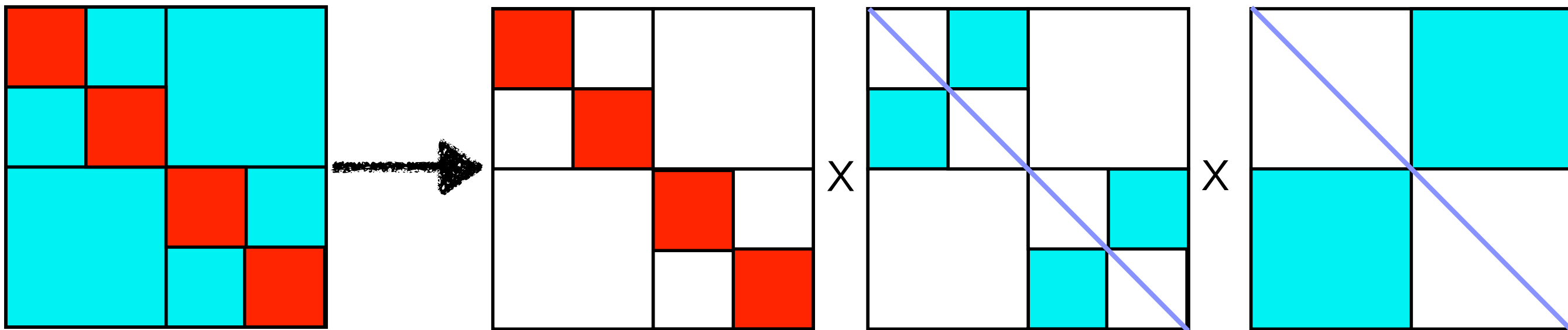
■ Full rank; ■ Low-rank; ■ Identity matrix; ■ Zero matrix;

Can we induct?



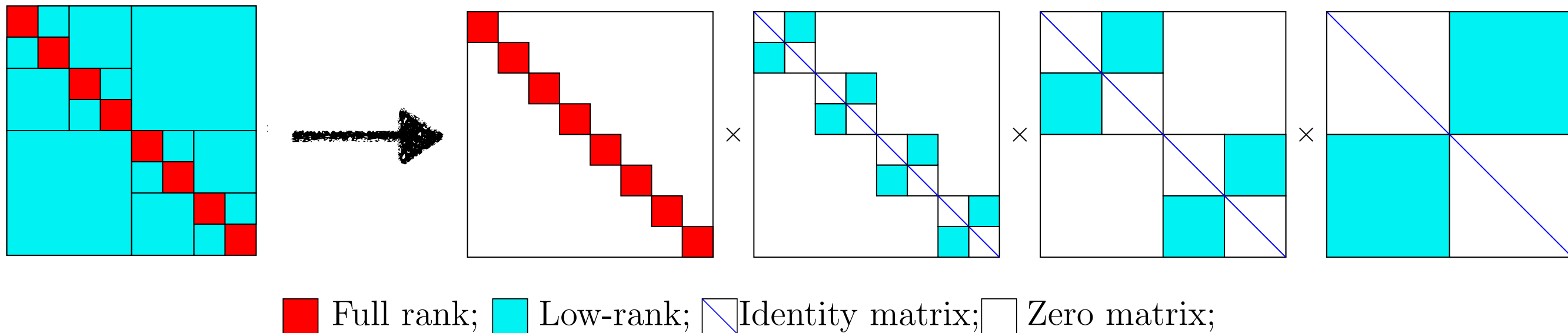
■ Full rank; ■ Low-rank; ▤ Identity matrix; □ Zero matrix;

Can we induct?



■ Full rank; ■ Low-rank; ▤ Identity matrix; □ Zero matrix;

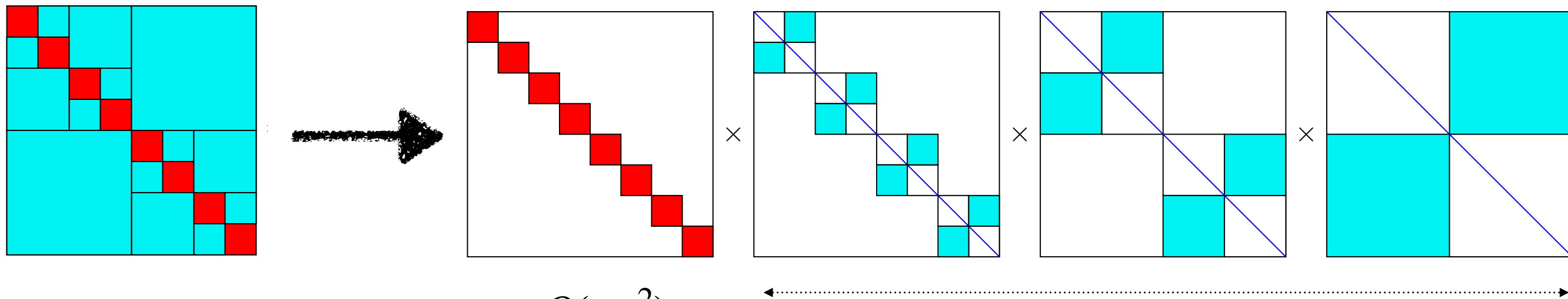
Algorithm and factorization costs



1. Compute all low-rank factorizations of off-diagonal blocks at all levels $O(n^2 r)$
2. Compute inverses of $n/p, p \times p$ matrices at the finest level $O(np^2)$
3. Loop over levels $j = \kappa - 1, \dots, 1$
 - a. Update the inverses of the coarser diagonal blocks
 - b. Update the off-diagonal low rank factors using the computed inverses $O(npr \log n)$

Factorization cost: $O(n^2 r + np^2 + npr \log^2 n)$

Apply/Inversion/Determinant cost



$$O(np^2)$$

Each matrix is of the form $(I + UV^T)$
where rank of U, V is $2r$

Cost of applying/inverting/computing
determinants at each level: $O(nr^2)$

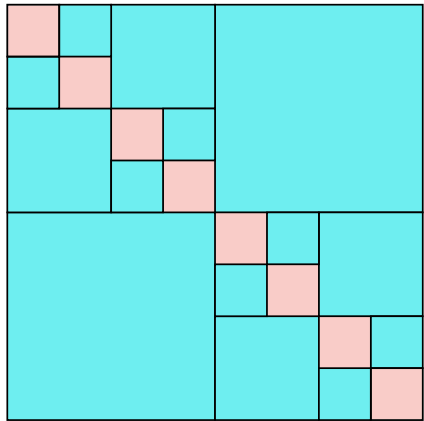
Post factorization

Inversion cost: $O(n)$

Determinant cost: $O(n)$

■ Full rank;
 ■ Low-rank;
 Identity matrix;
 Zero matrix;

Low rank factorizations of off-diagonal blocks



■ Full rank; ■ Low-rank; ▢ Identity matrix; □ Zero matrix;

Compute all low-rank factorizations of off-diagonal blocks at all levels

$O(n^2 r)$

Options:

1. Analysis

- Compute analytical low rank decompositions of the kernel

$$\bullet \quad e^{-(s-t)^2/2} = \sum_{n=0}^r \frac{(t-c)^n}{n!} h_n(s-c) + O(\epsilon)$$

Need different expansions per kernel!

2. Linear algebra

- Partial pivoted LU
- Adaptive Cross Approximation

Can be unstable sometimes!

3. Using nested basis

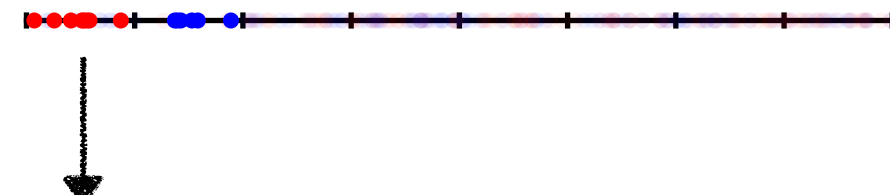
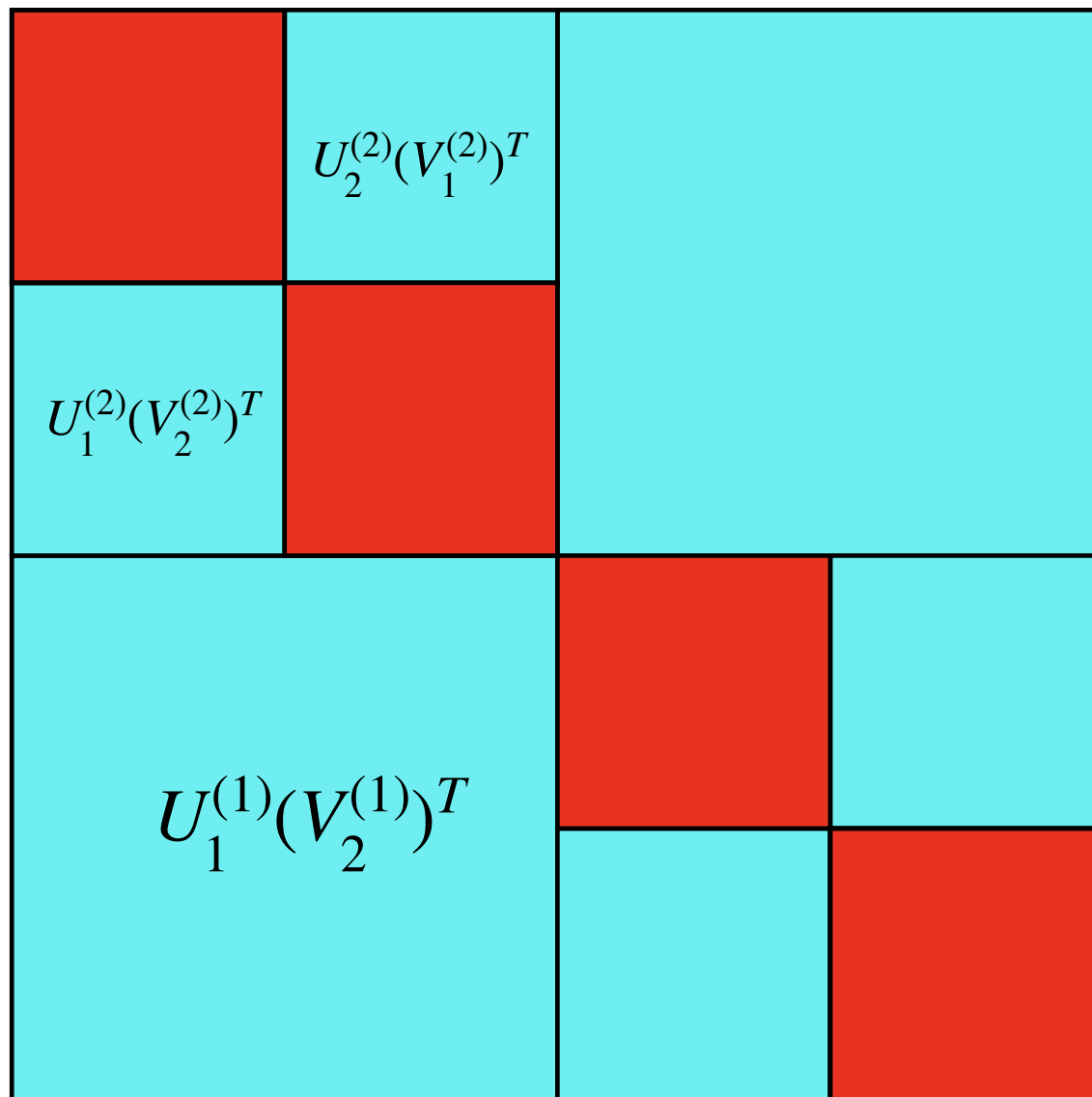
Can be unstable sometimes - but instabilities known!

Nested basis

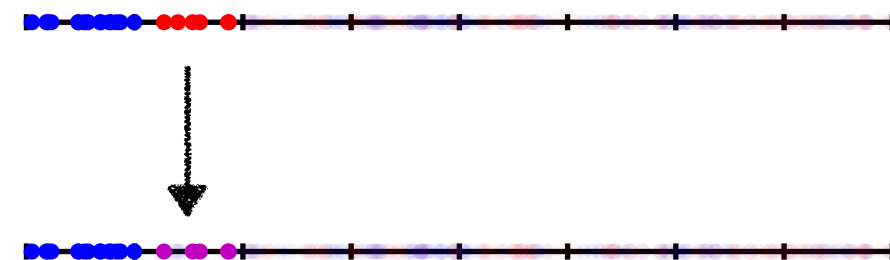
Q: Given $U_1^{(2)}$, $U_2^{(2)}$, can we compute $U_1^{(1)}$?

In general: No!

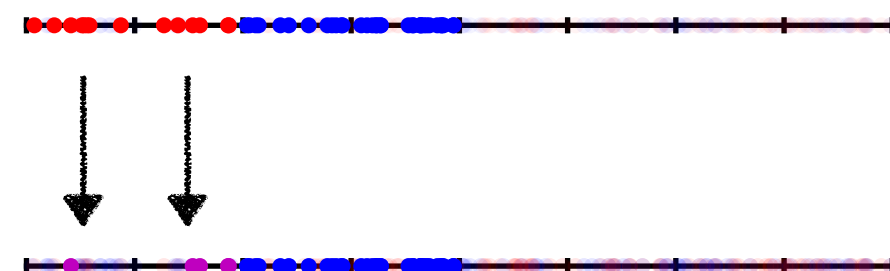
$U_i^{(j)}$ can be thought of as subset of columns of original matrix



$U_1^{(2)}$



$U_2^{(2)}$



$U_1^{(1)}$

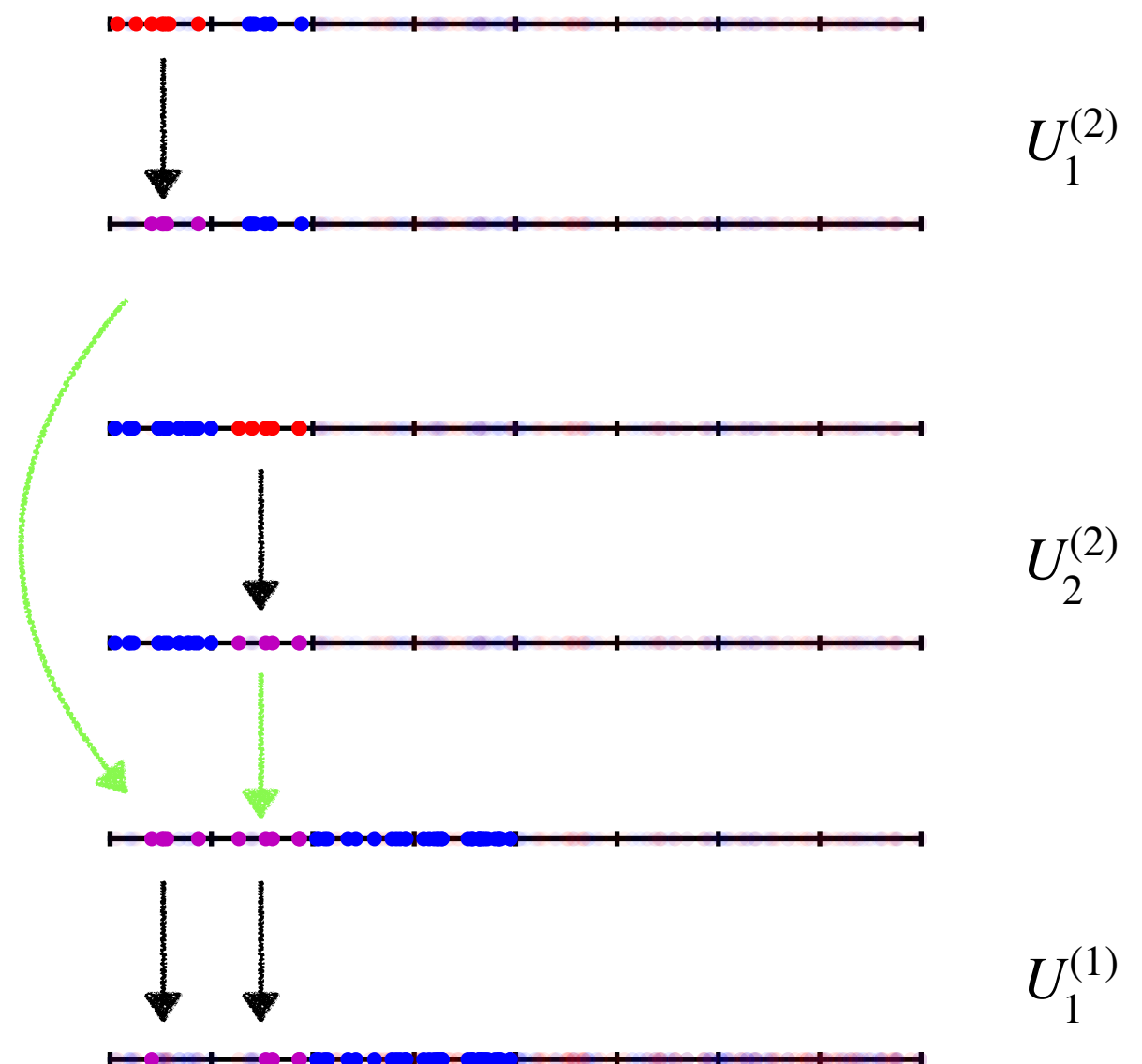
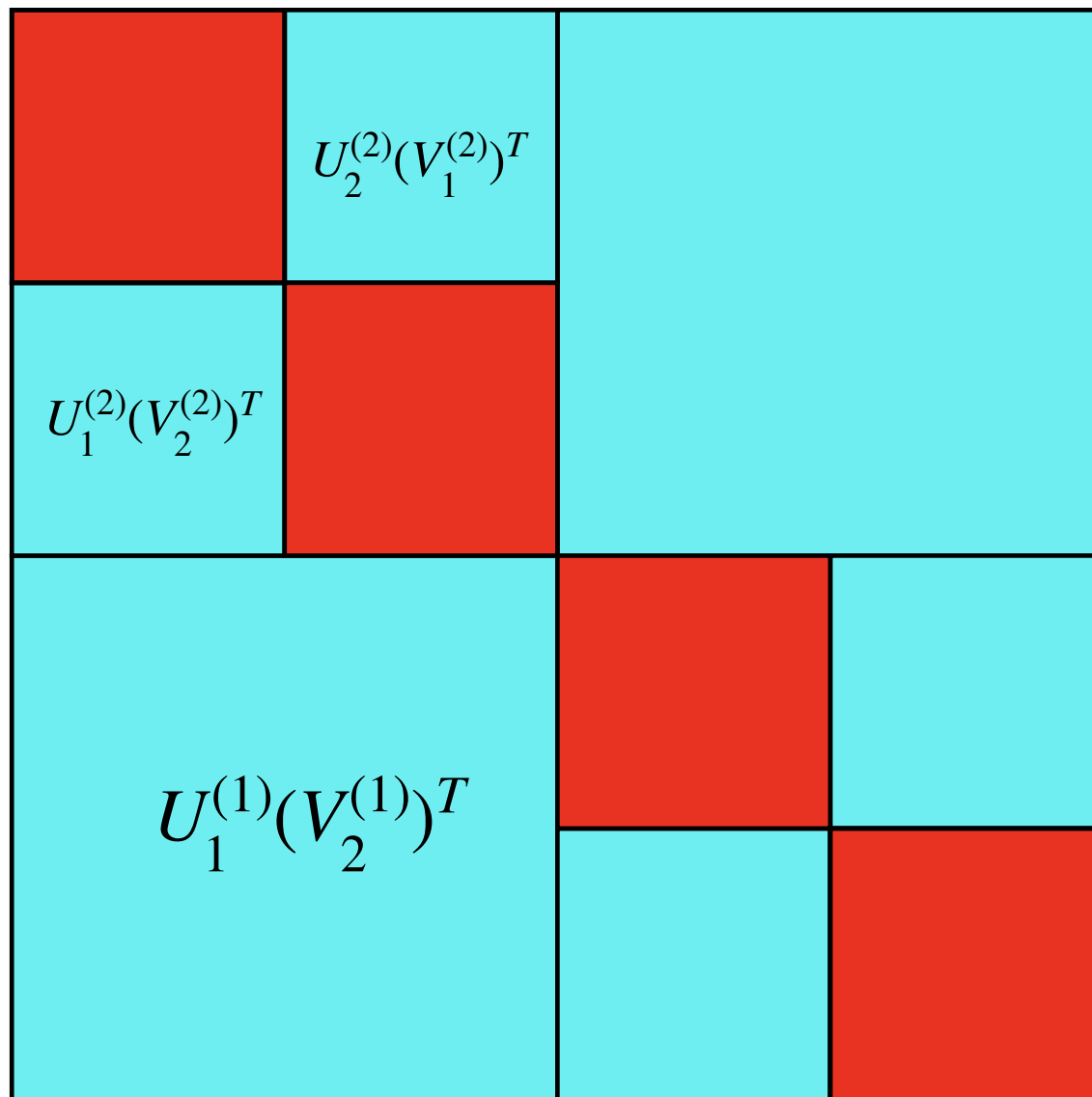
Nested basis

Q: Given $U_1^{(2)}$, $U_2^{(2)}$, can we compute $U_1^{(1)}$?

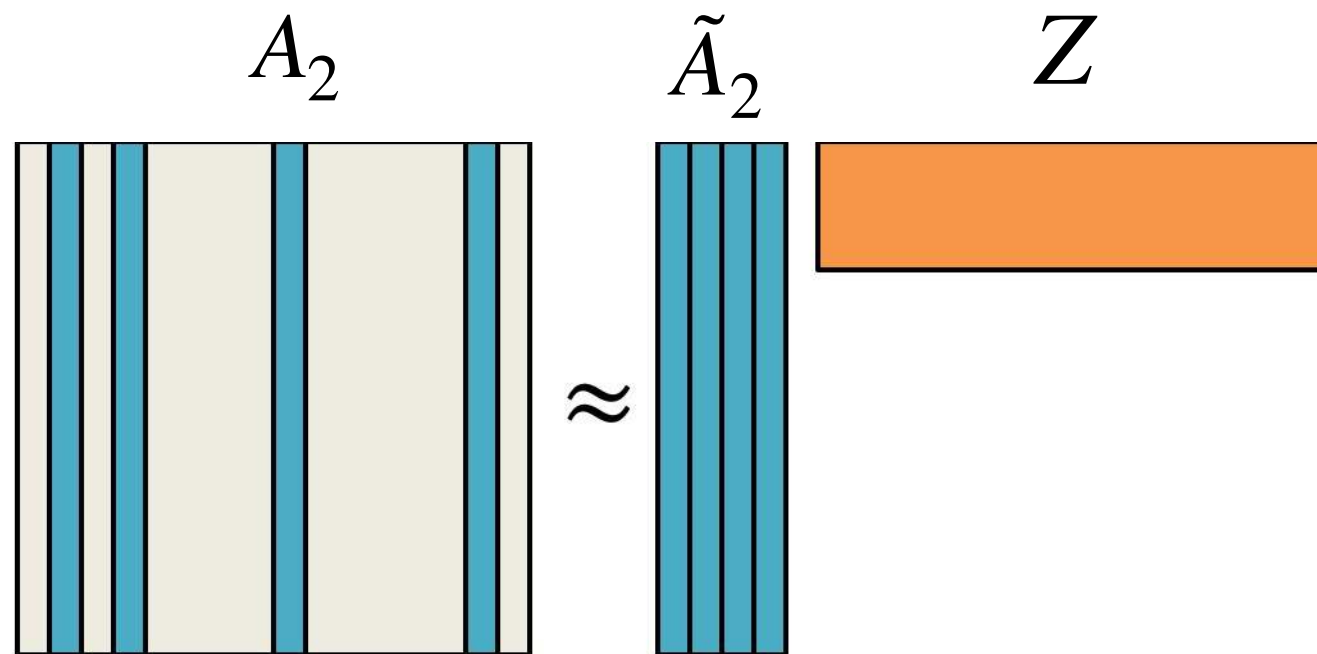
In general: No!

Even if you can, factorization cost still $O(n^2)$

$U_i^{(j)}$ can be thought of as subset of columns of original matrix



Interpolative Decomposition (ID)



$$\|A_2 - \tilde{A}_2 Z\| \leq (1 + \sqrt{k(n-k)})s_{k+1}$$

$$|Z_{i,j}| \leq 1$$

Combinatorial search, exponential cost

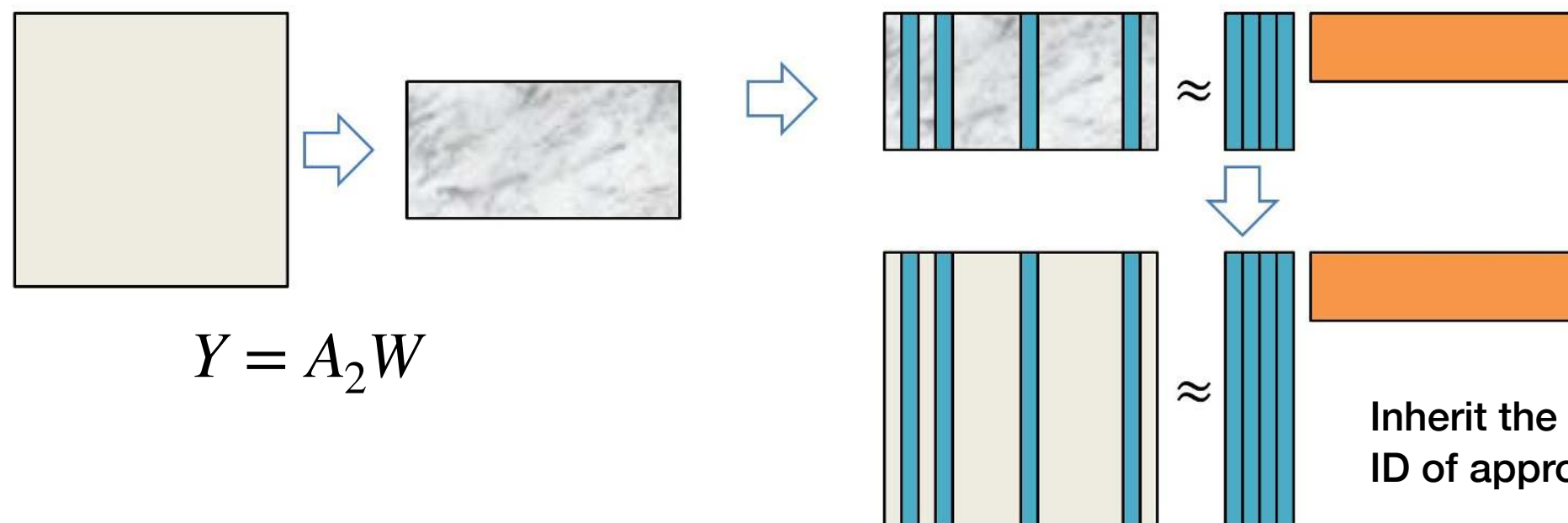
$$\|A_2 - \tilde{A}_2 Z\| \leq (1 + \sqrt{k(n-k)})s_{k+1}$$

$$|Z_{i,j}| \leq 2$$

$$O(N \cdot p^2 \log N)$$

In practice, rank revealing QR works fine

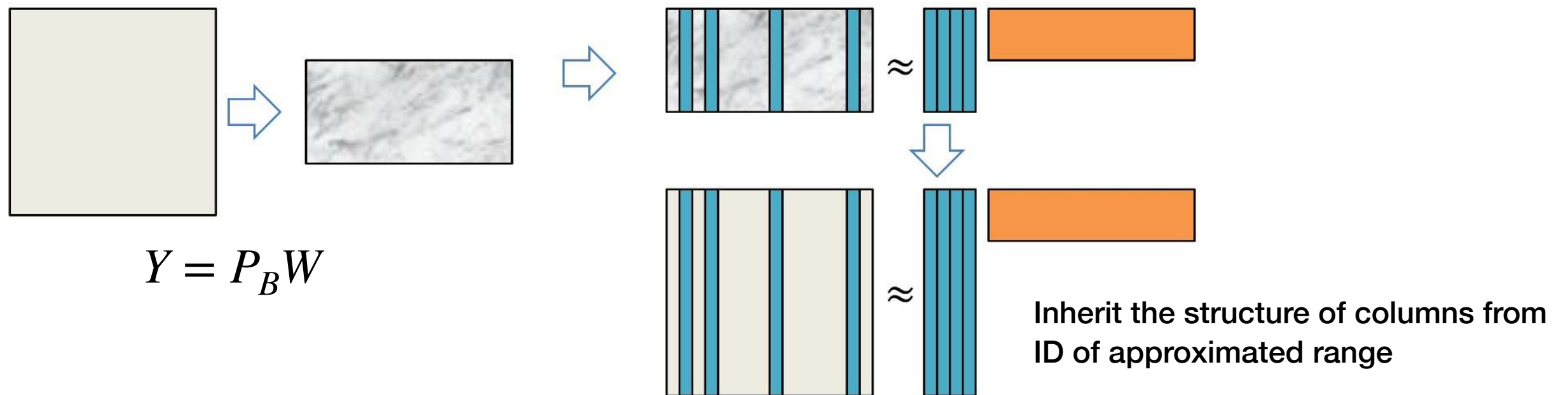
Low rank approximation that uses columns of input matrix



Inherit the structure of columns from ID of approximated range

Randomized approach for computing ID

Interpolative Decomposition (ID)



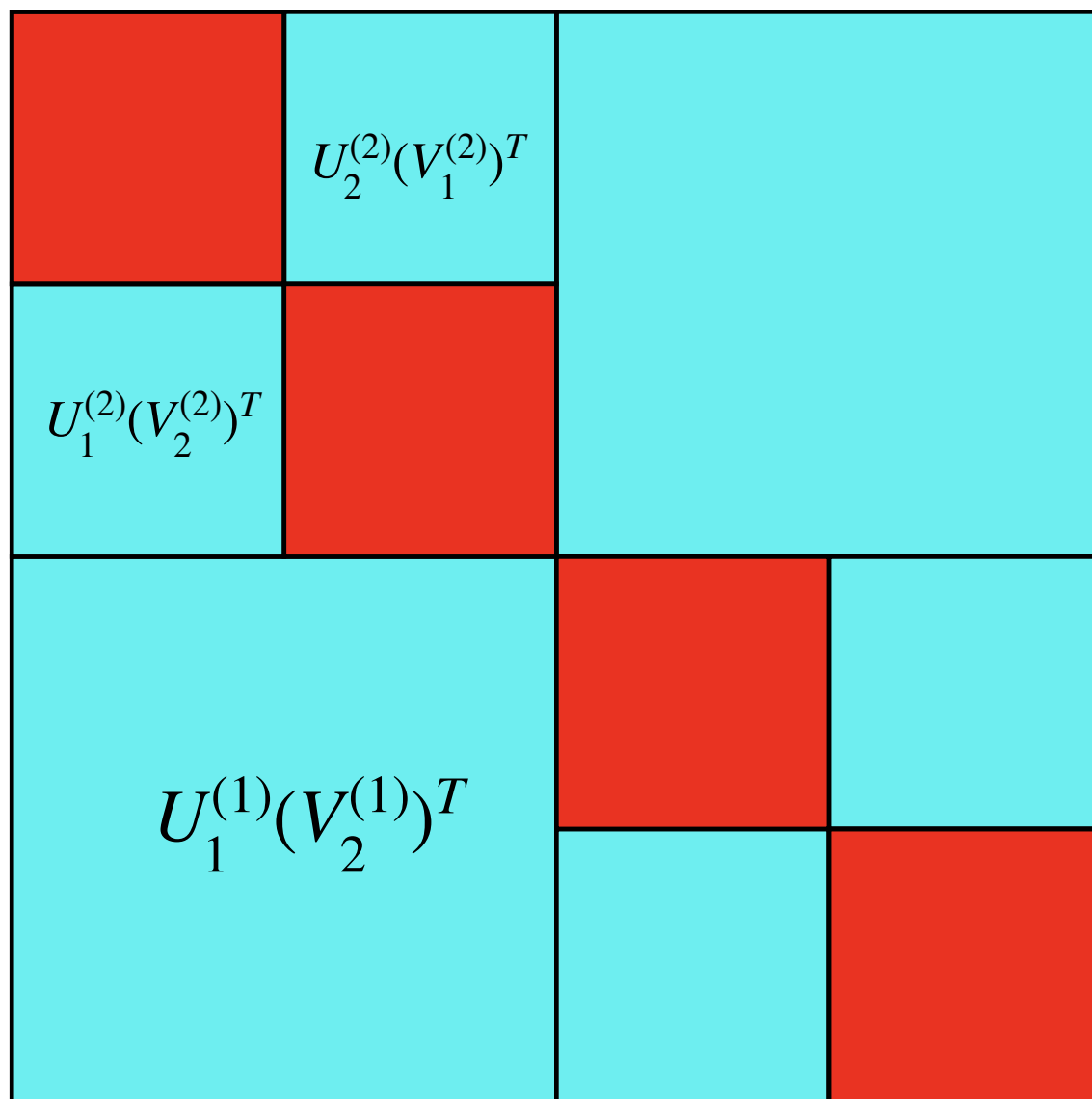
Randomized approach for computing ID

Issues:

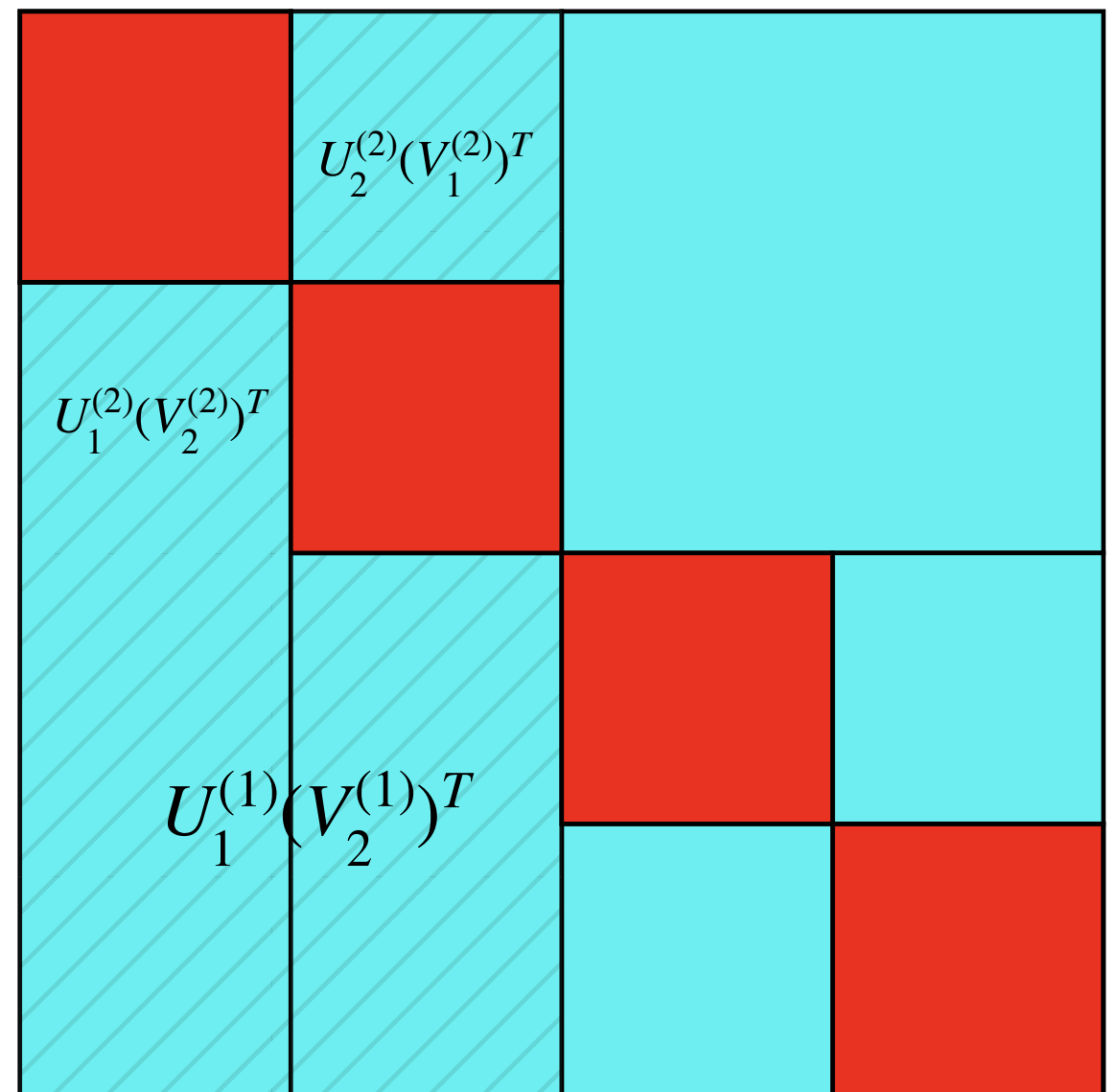
~~When using in a hierarchical manner, lack of interpretability after first compression~~

Entries post compression are sub-blocks of the original matrix

HODLR vs recursive skeletonization

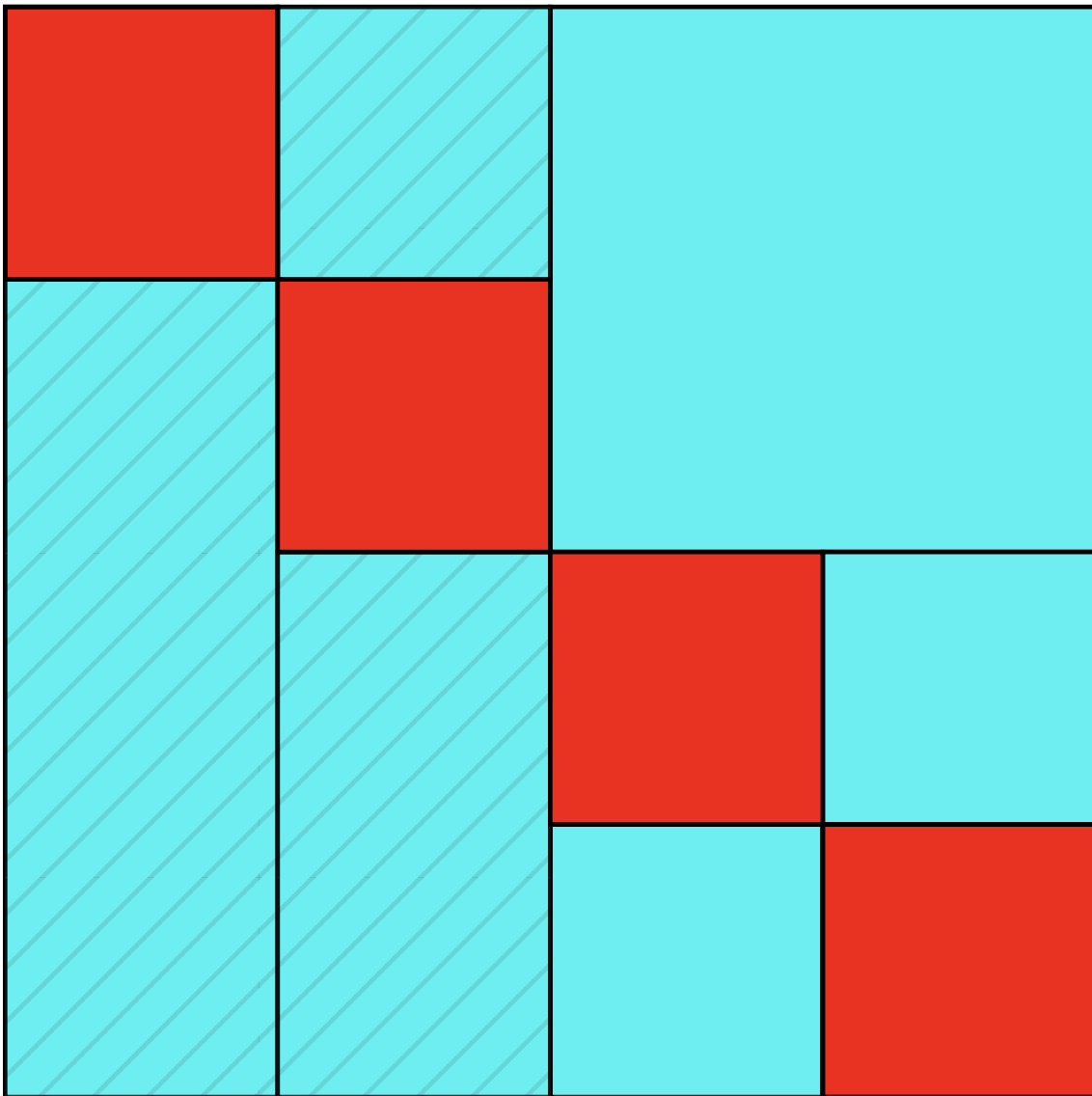


HODLR



**Recursive
Skeletonization**

Recursive skeletonization



Key idea: Instead of compressing off-diagonal blocks, use interpolative decomposition to eliminate redundant points

Suppose $A_{1,2:4}$, and $A_{2:4,1}$ are low-rank

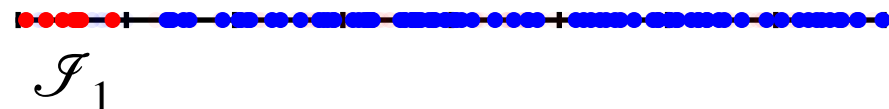
$$\begin{bmatrix} A_{1,1} & A_{1,2:4} \\ A_{2:4,1} & A_{2:4,2:4} \end{bmatrix}$$

Let $\mathcal{J}_1$ be the index set of points in interval 1

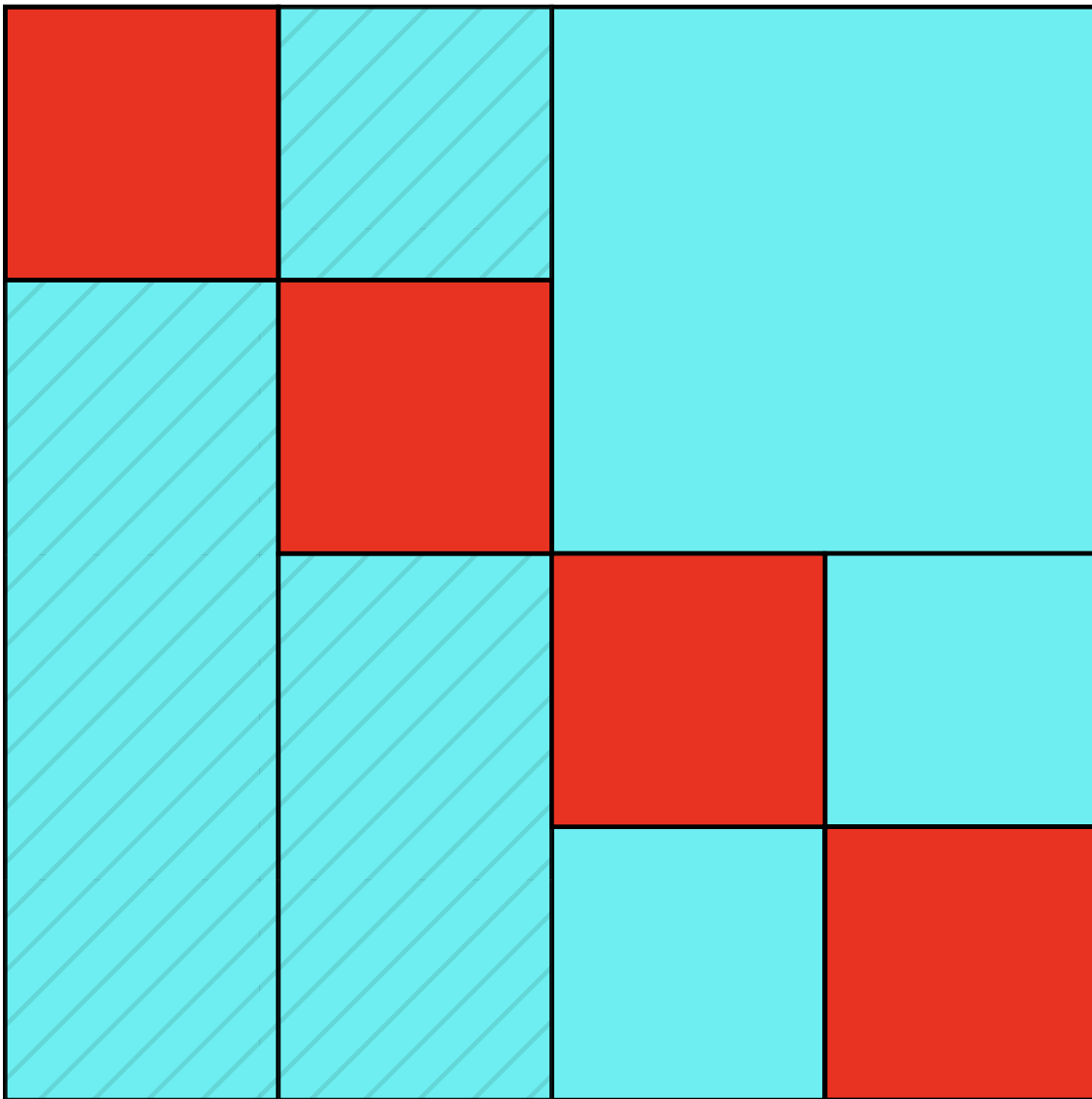
Since $A_{1,2:4}$, $A_{2:4,1}$ are low rank, we can construct the interpolative decomposition of

$$\begin{bmatrix} A_{2:4,1} \\ A_{1,2:4}^* \end{bmatrix}$$

$2(n - |\mathcal{J}_1|) \times |1s|$



Recursive skeletonization



Key idea: Instead of compressing off-diagonal blocks, use interpolative decomposition to eliminate redundant points

Suppose $A_{1,2:4}$, and $A_{2:4,1}$ are low-rank

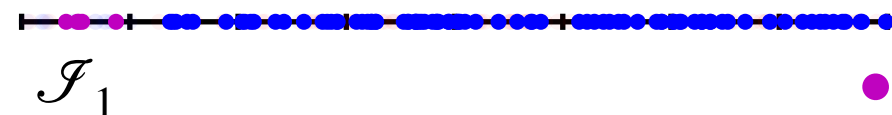
$$P \begin{bmatrix} A_{1,1} & A_{1,2:4} \\ A_{2:4,1} & A_{22} \end{bmatrix} \rightarrow \begin{bmatrix} A_{1r,1r} & A_{1r,1s} & A_{1r,2:4} \\ A_{1s,1r} & A_{1s,1s} & A_{1s,2:4} \\ A_{2:4,1r} & A_{2:4,1s} & A_{2:4,2:4} \end{bmatrix}$$

Let $\mathcal{J}_1$ be the index set of points in interval 1

Since $A_{1,2:4}$, $A_{2:4,1}$ are low rank, we can construct the interpolative decomposition of

$$P \begin{bmatrix} A_{2:4,1} \\ A_{1,2:4}^* \end{bmatrix} = \begin{bmatrix} A_{2:4,1s} \\ A_{1s,2:4}^* \end{bmatrix} \begin{bmatrix} I_{|1s|} & T_{1,sr} \end{bmatrix}$$

$2(n - |\mathcal{J}_1|) \times |\mathcal{J}_1| \quad 2(n - |\mathcal{J}_1|) \times |1s| \quad |1s| \times |1r|$



● Skeleton points, $1s$

Recursive skeletonization

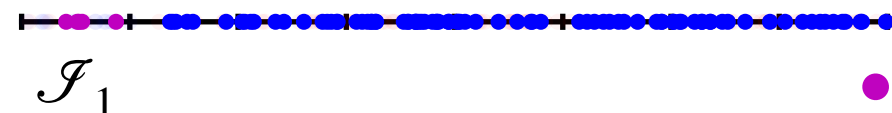
Key idea: Instead of compressing off-diagonal blocks, use interpolative decomposition to eliminate redundant points

Eliminate $A_{1r,2:4}$, and $A_{2:4,1r}$ are low-rank

$$Q_1^* \begin{bmatrix} A_{1r,1r} & A_{1r,1s} & A_{1r,2:4} \\ A_{1s,1r} & A_{1s,1s} & A_{1s,2:4} \\ A_{2:4,1r} & A_{2:4,1s} & A_{2:4,2:4} \end{bmatrix} Q_1^* = \begin{bmatrix} B_{1r,1r} & B_{1r,1s} & 0 \\ B_{1s,1r} & A_{1s,1s} & A_{1s,2:4} \\ 0 & A_{2:4,1s} & A_{2:4,2:4} \end{bmatrix}$$

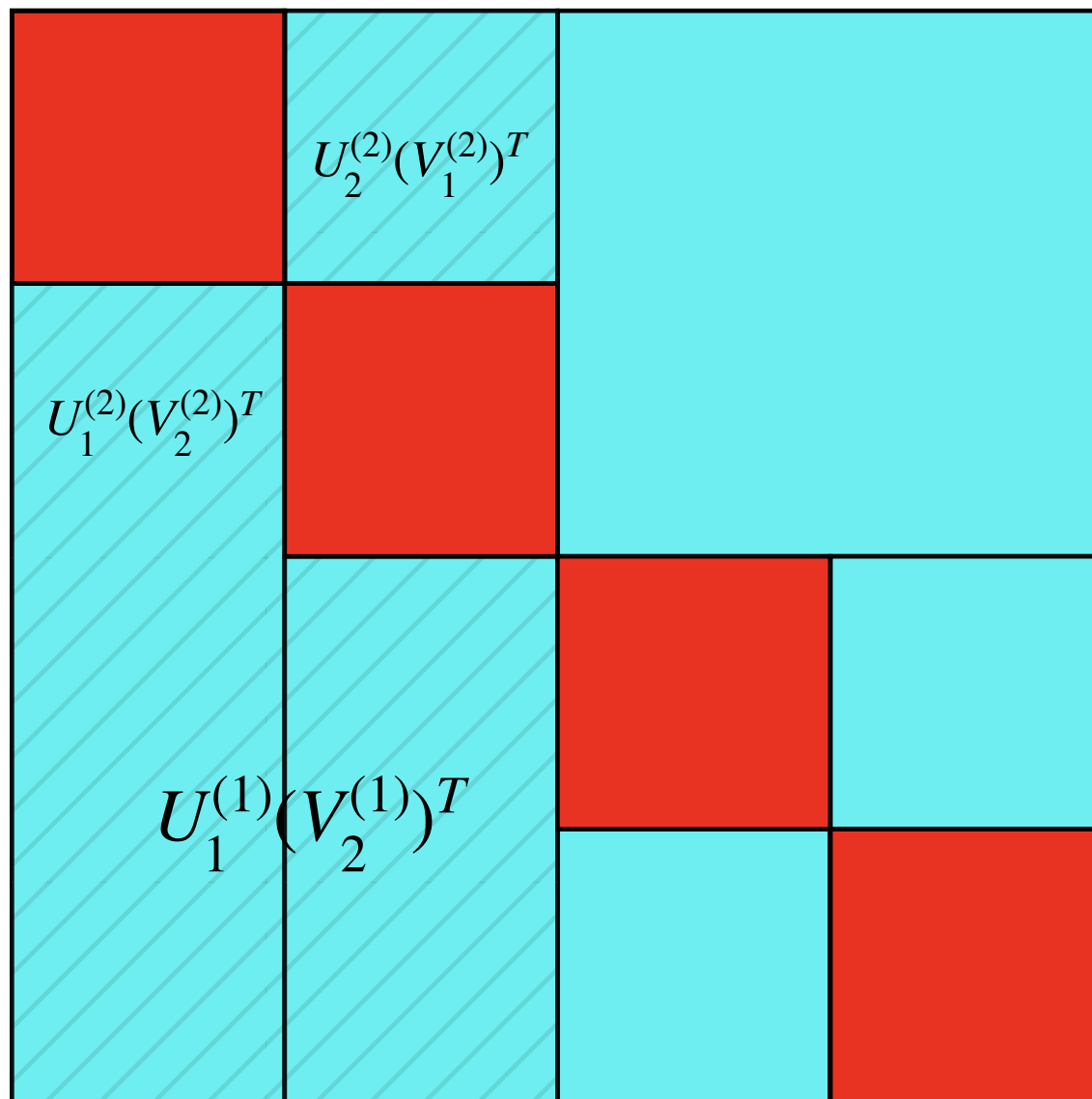
where

$$Q_1 = \begin{bmatrix} I & 0 & 0 \\ -T_{1,sr} & I & 0 \\ 0 & 0 & I \end{bmatrix}$$

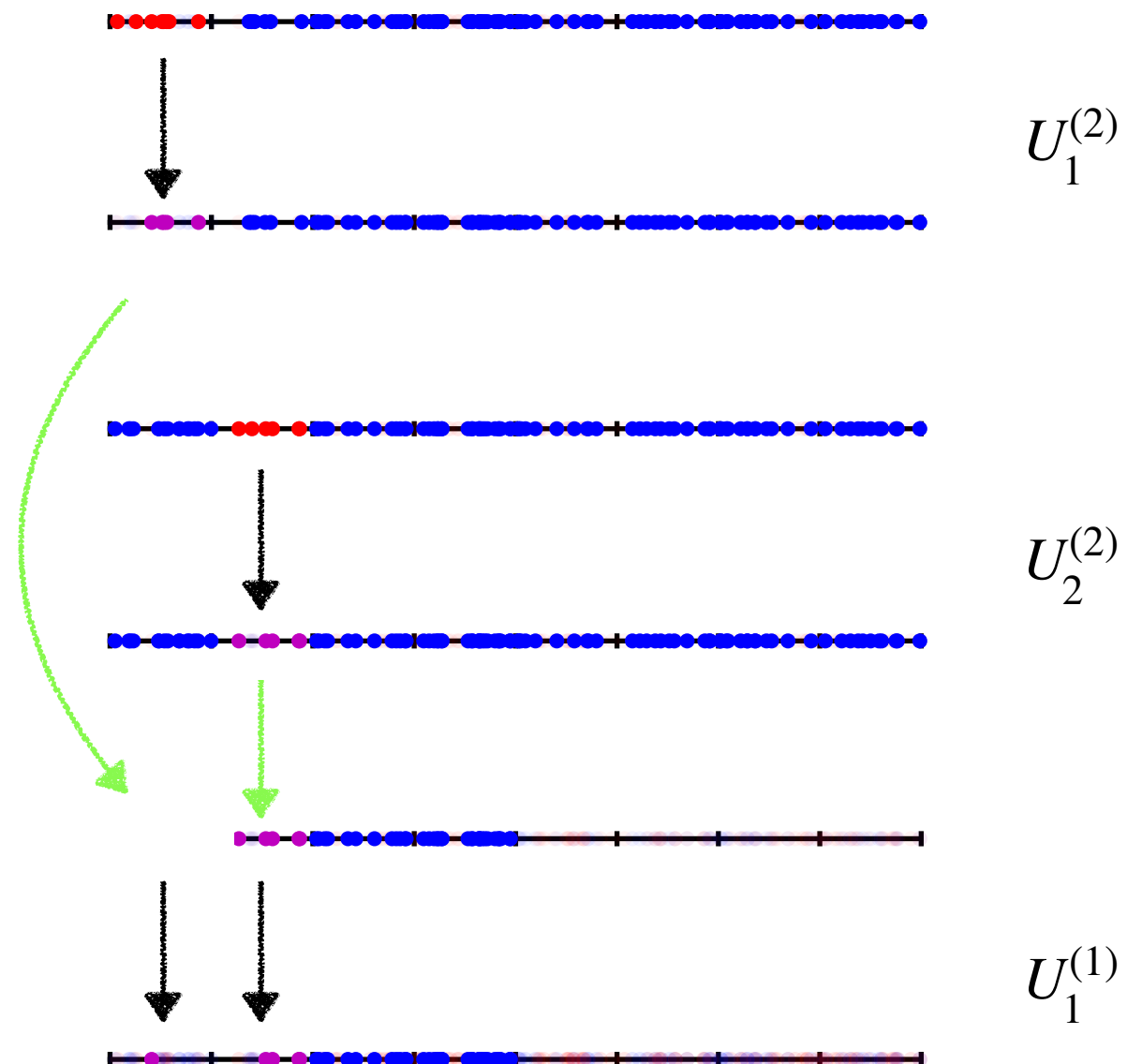


● Skeleton points, $1s$

Recursive skeletonization



Key idea: Instead of compressing off-diagonal blocks, use interpolative decomposition to eliminate redundant points

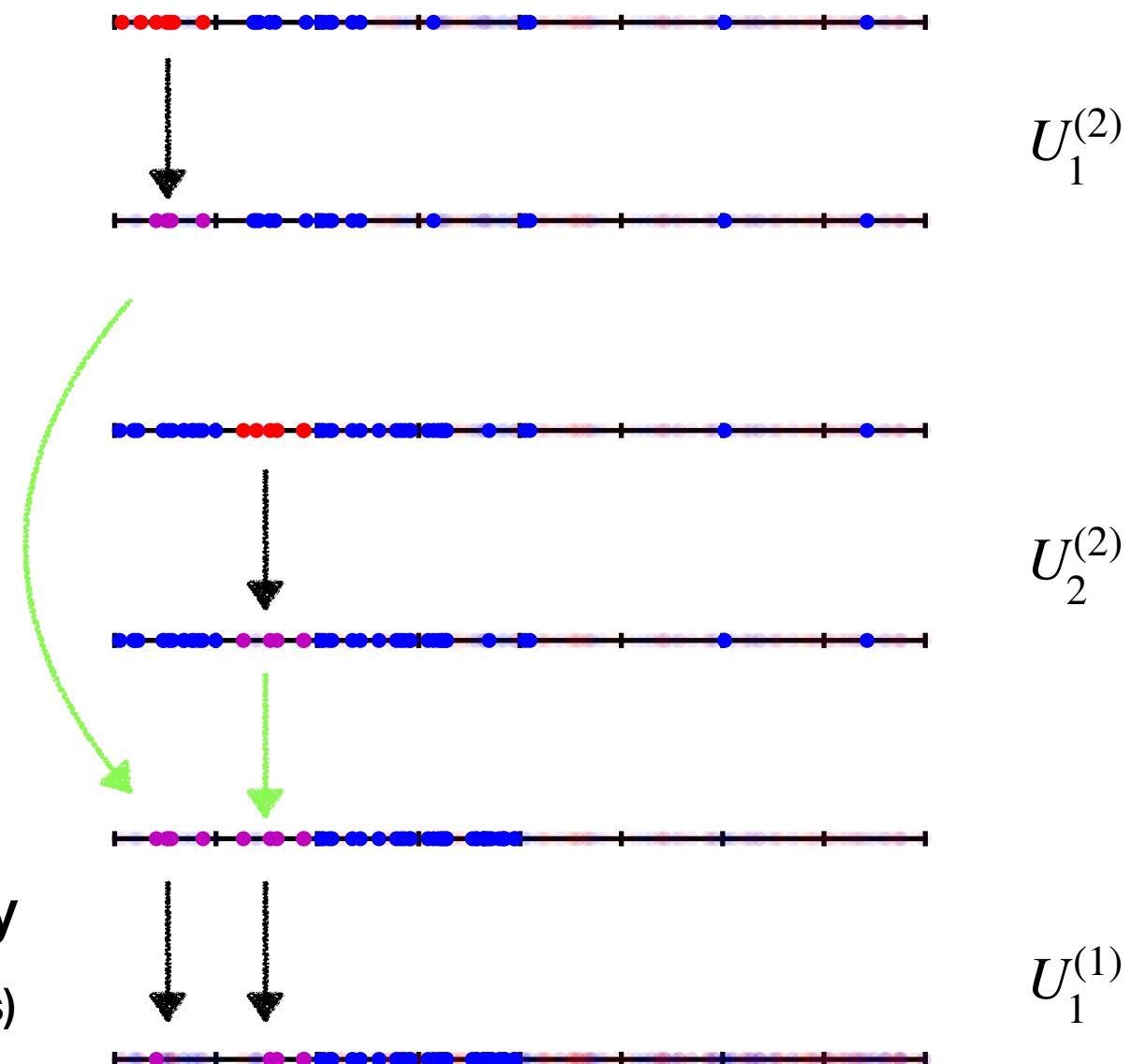
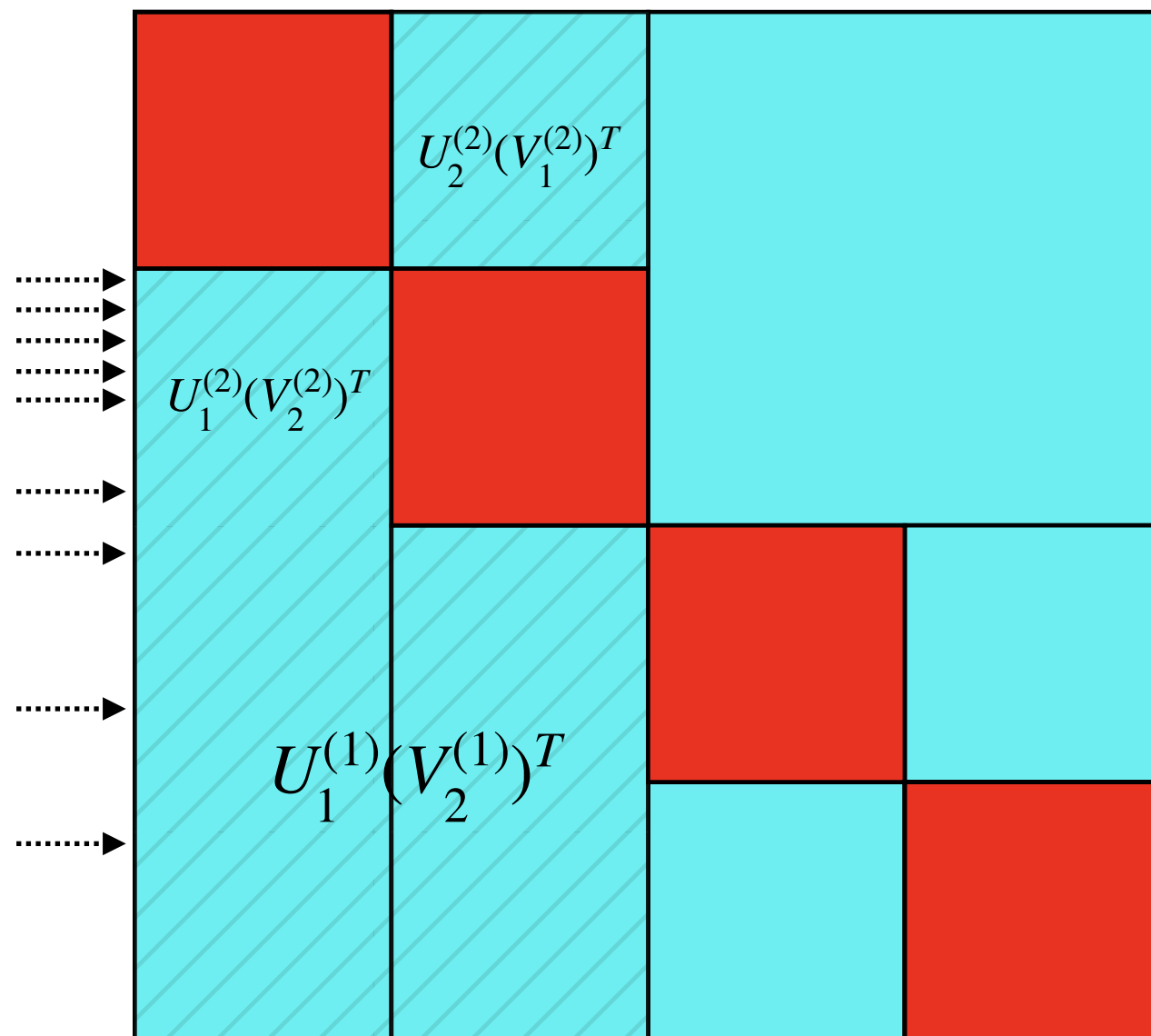


Nested basis

Q: Given $U_1^{(2)}$, $U_2^{(2)}$, can we compute $U_1^{(1)}$?

Use proxy points! Factorization cost $O(n)$

$U_i^{(j)}$ can be thought of as subset of columns of original matrix



Proofs for kernels satisfying Green's identity

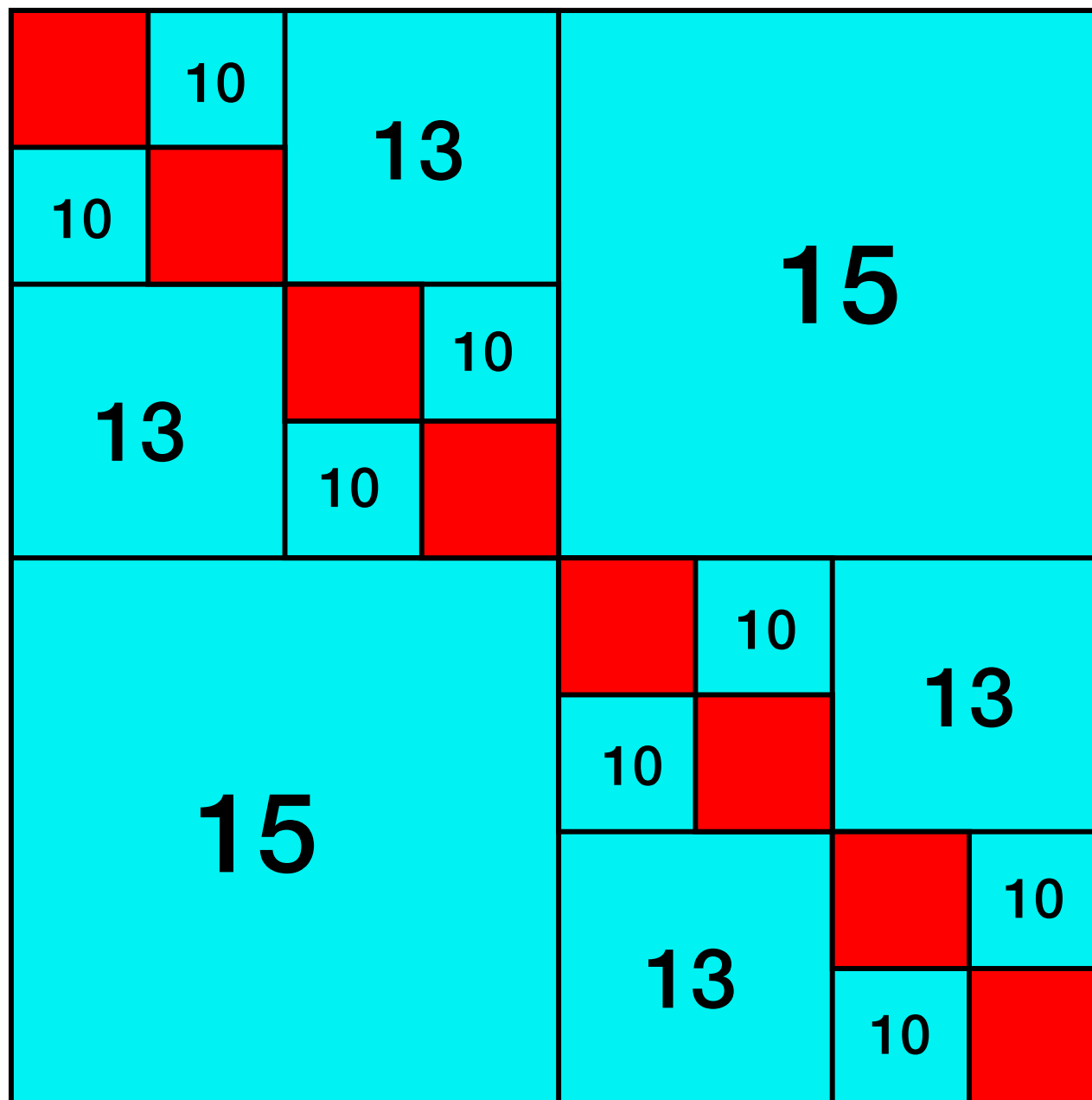
Heuristic works for larger family of kernels (e.g. Gaussians)

The need for order

Computational Task:

$$\operatorname{argmax}_{\theta} \mathcal{L}_{\theta} \propto \frac{1}{\det C(t; \theta)^{1/2}} e^{-\frac{1}{2} y^T C^{-1}(t; \theta) y}$$

$$C(t, \theta) = \sigma_{\varepsilon}^2 I + K(t, t'; \theta)$$



$$K(t, t', \theta) = \theta(0) + \frac{1}{\sqrt{\theta(1) + (t - t')^2}}$$

$$\theta(0) = 1, \theta(1) = 0.01$$



10k t'_i s uniformly distributed on $[0,1]$

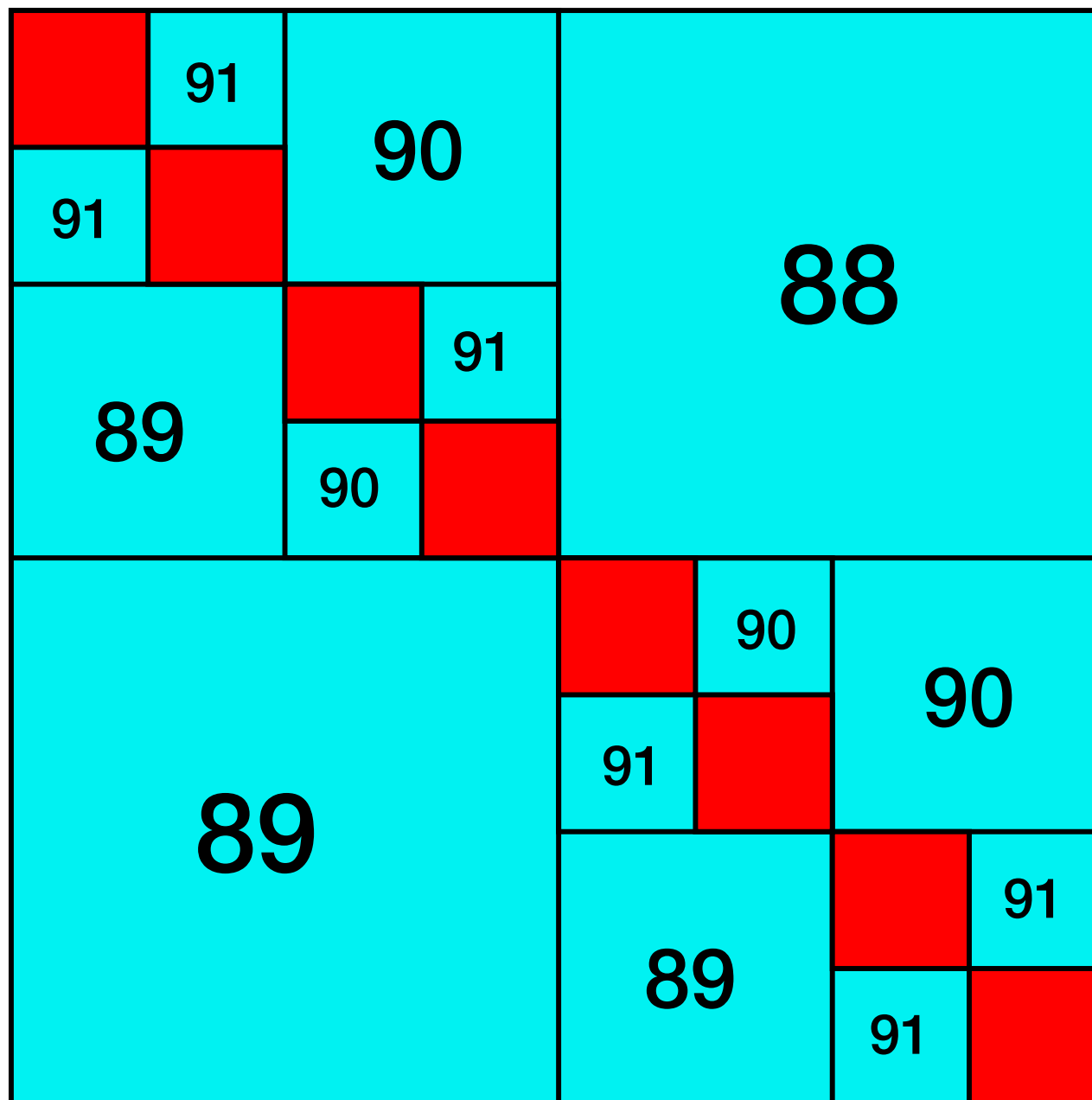
Rank structure exists only
if points are sorted

The need for order

Computational Task:

$$\operatorname{argmax}_{\theta} \mathcal{L}_{\theta} \propto \frac{1}{\det C(t; \theta)^{1/2}} e^{-\frac{1}{2} y^T C^{-1}(t; \theta) y}$$

$$C(t, \theta) = \sigma_{\varepsilon}^2 I + K(t, t'; \theta)$$



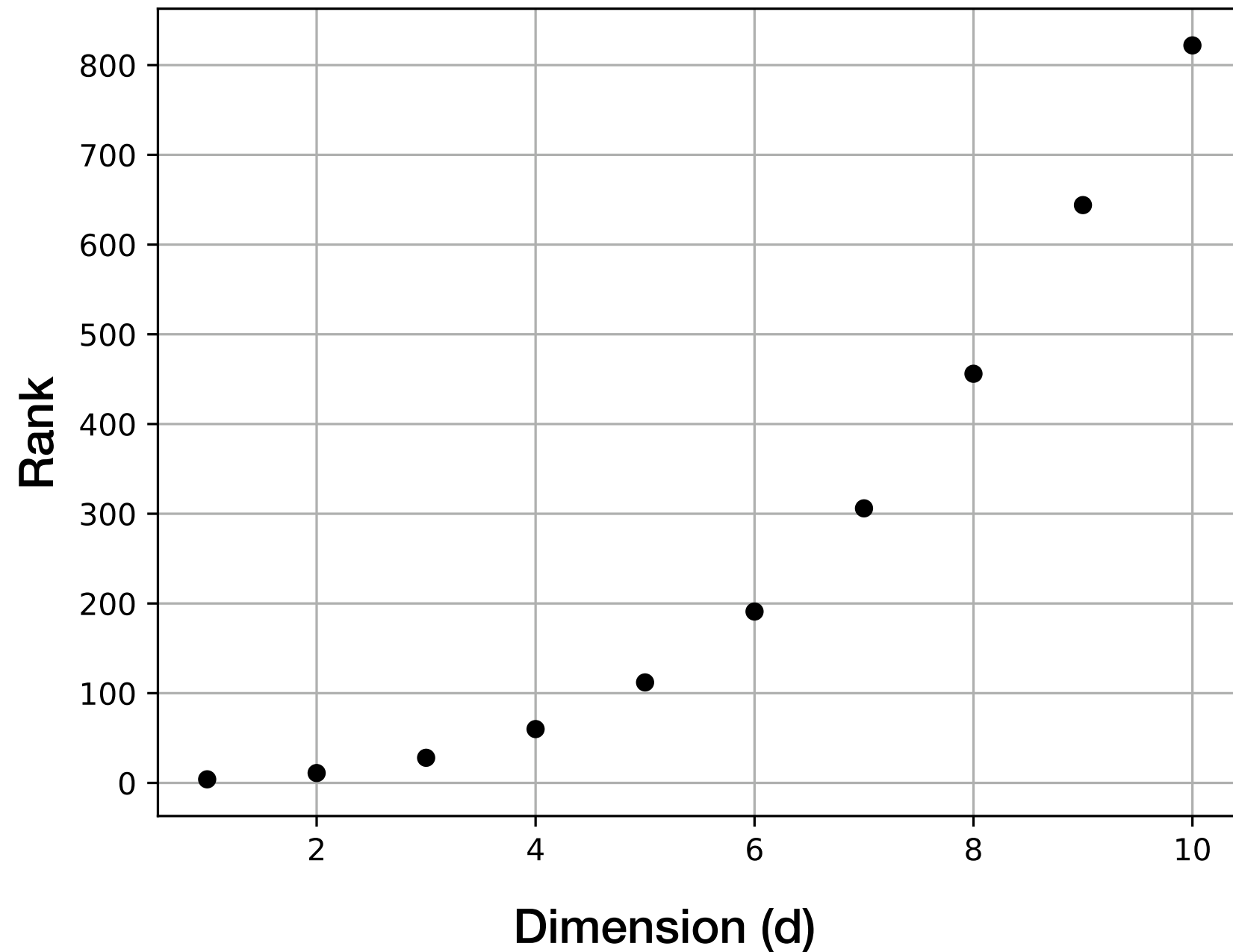
$$K(t, t', \theta) = \theta(0) + \frac{1}{\sqrt{\theta(1) + (t - t')^2}}$$

$$\theta(0) = 1, \theta(1) = 0.01$$



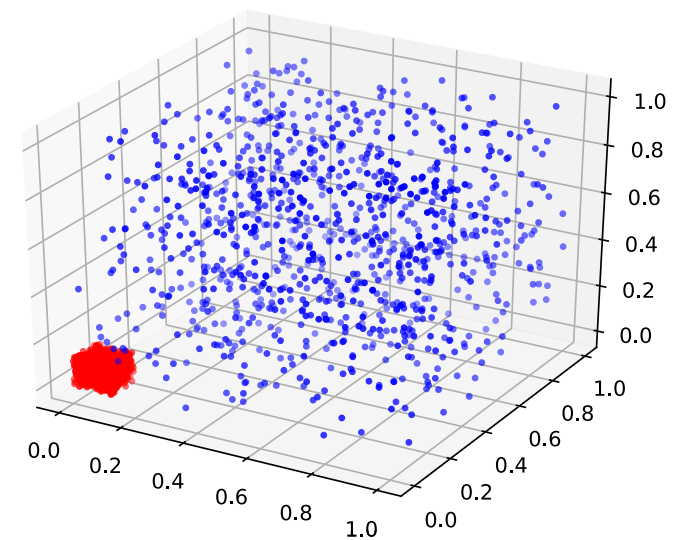
10k t'_i s uniformly distributed on $[0, 1]$

The curse of dimensionality



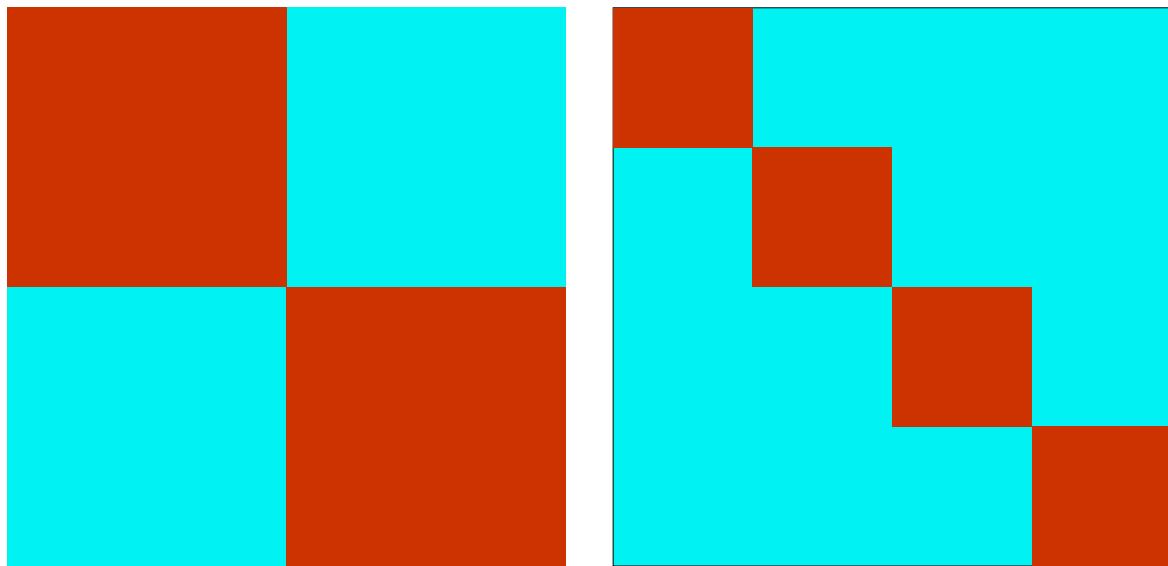
$$K(t, t', \theta) = \theta(0) + e^{-\frac{|t - t'|^2}{2\theta(1)^2}}$$
$$\theta(0) = 1, \theta(1) = 3$$

$$t, t' \in \mathbb{R}^d, \quad t' \in [0, 0.125]^d, \quad t \in [0, 1]^d \setminus [0, 0.125]^d$$

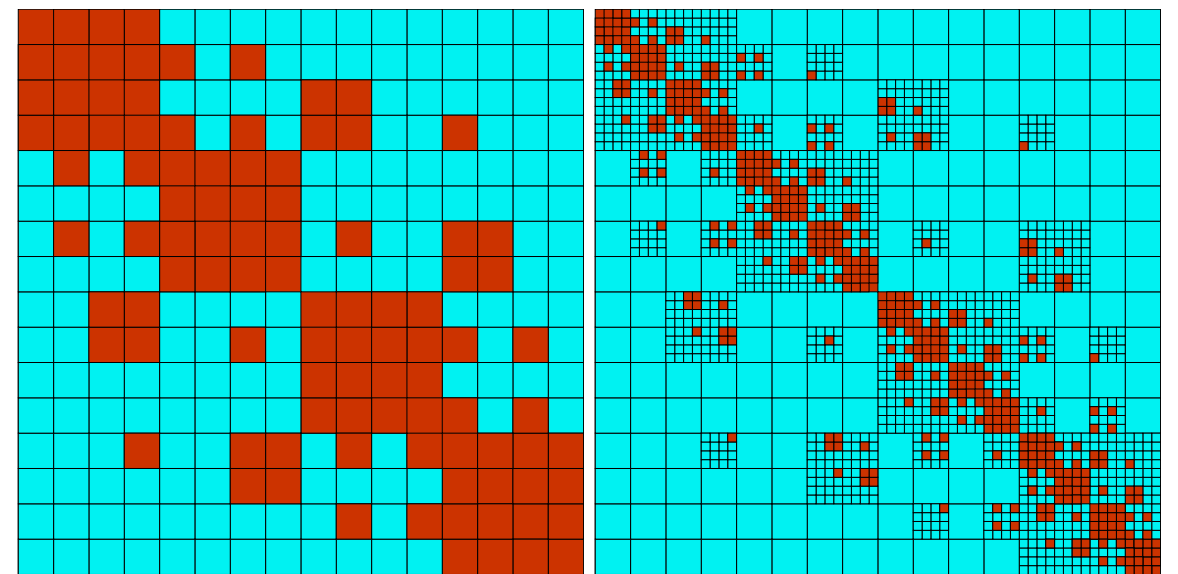


Sample distribution in 3d

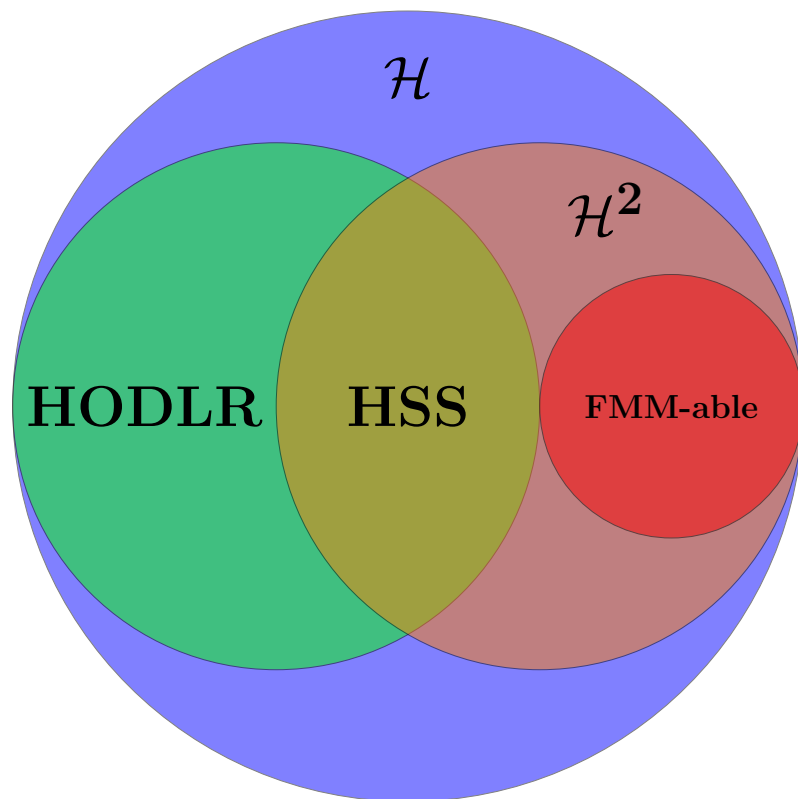
The zoo of matrix factorizations



HODLR/HSS matrices



FMM/ $\mathcal{H}^2$ matrices

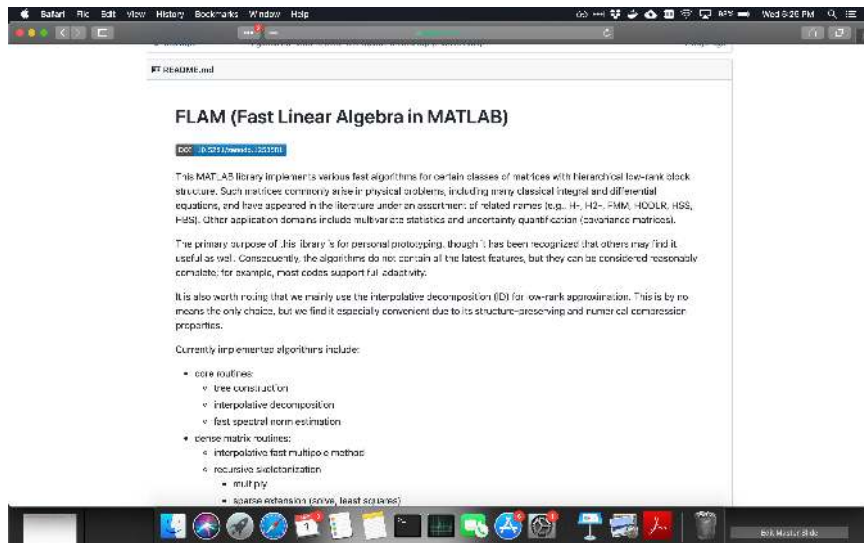


Butterfly/FFT matrices

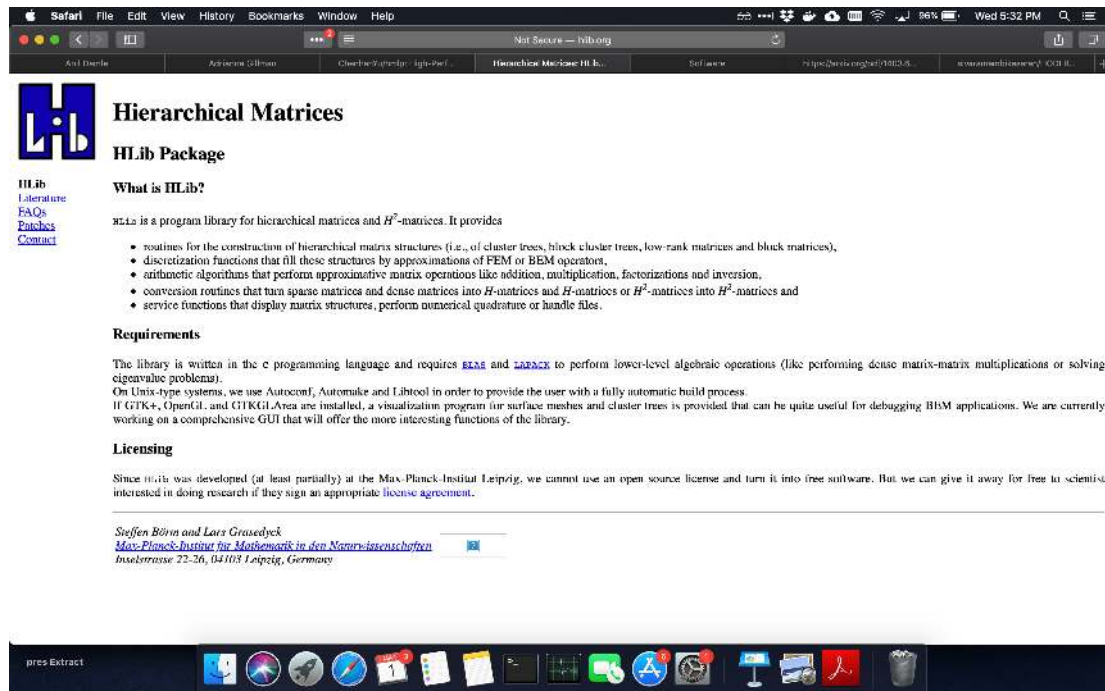
Low-rank structure
↓

	Nested basis →	
	No	Yes
Strong	HODLR	HSS
Weak	$\mathcal{H}$	$\mathcal{H}^2$

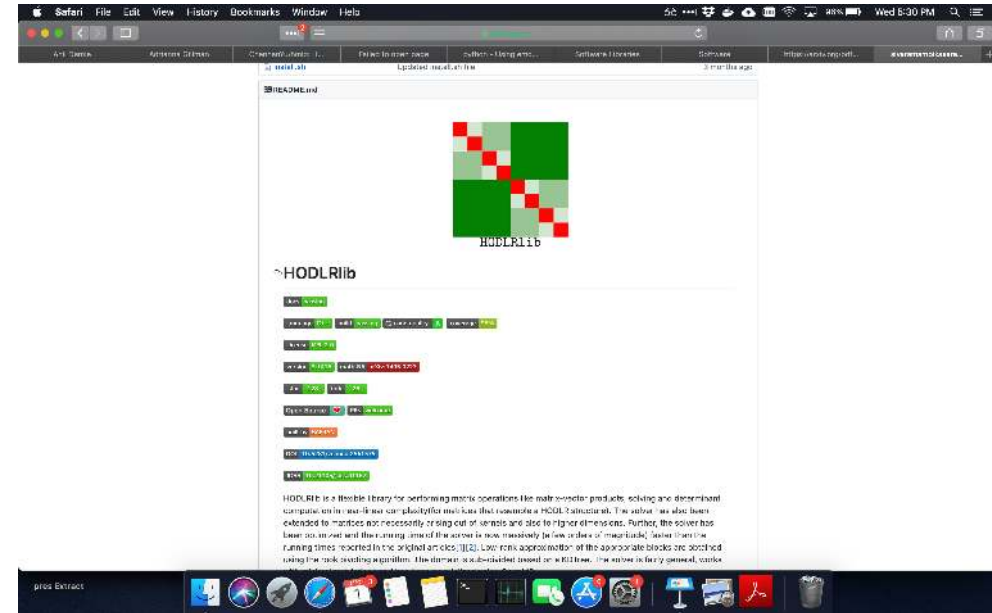
Software



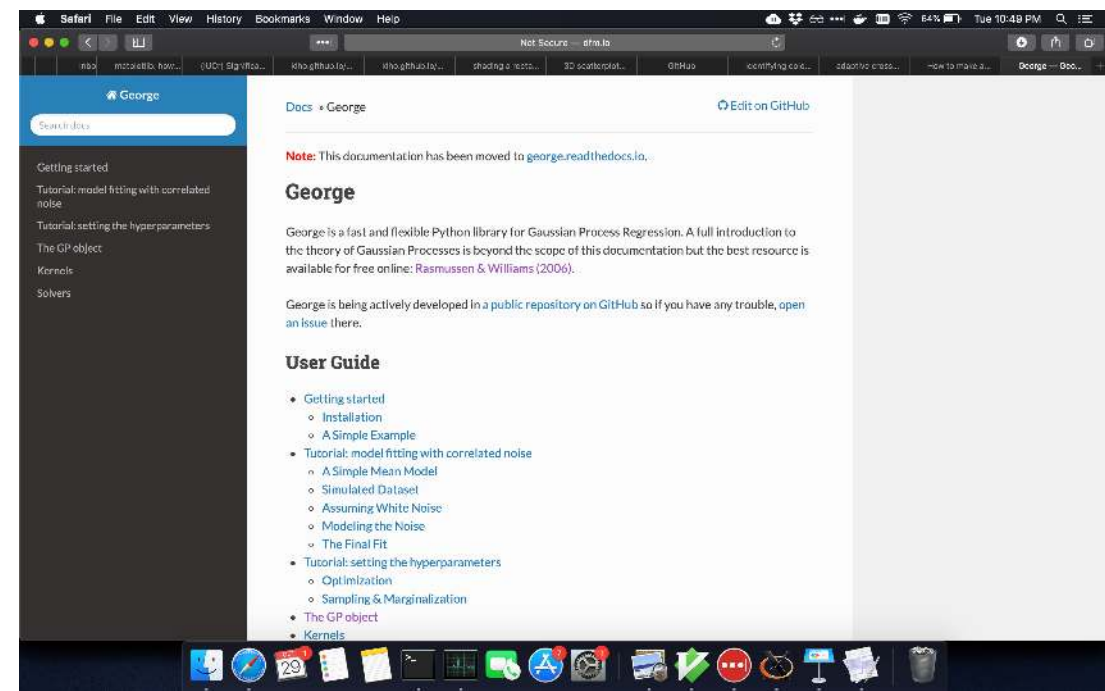
<https://github.com/klho/FLAM>



<http://www.hlib.org>

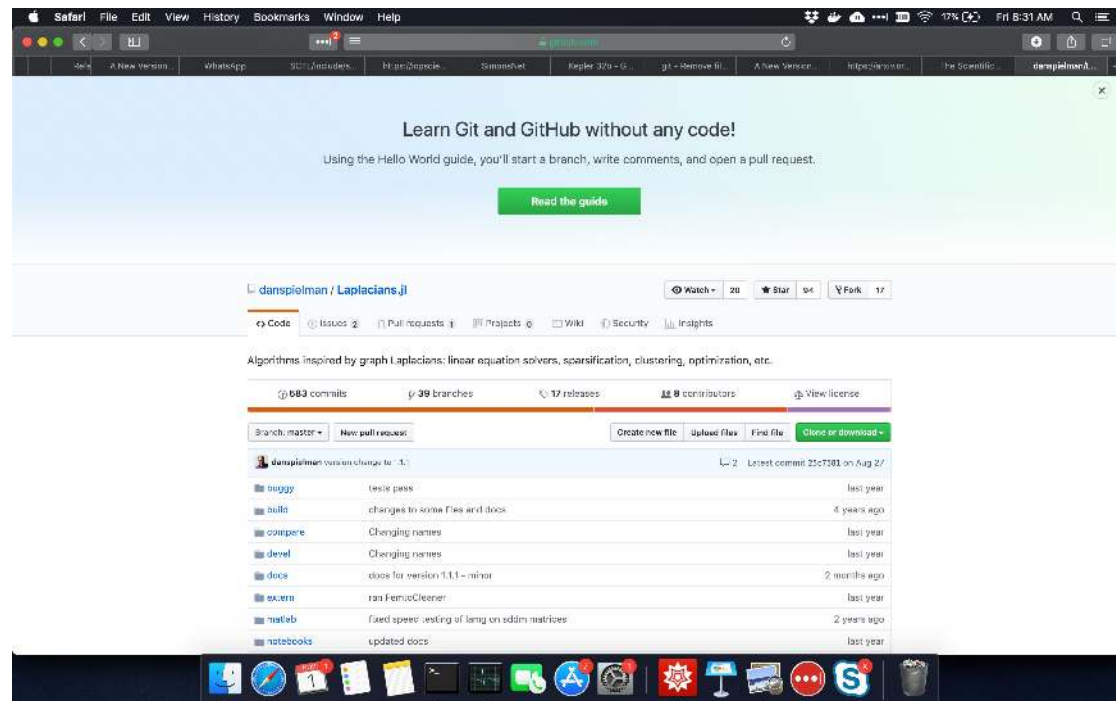


<https://github.com/sivaramambikasaran/HODLR>

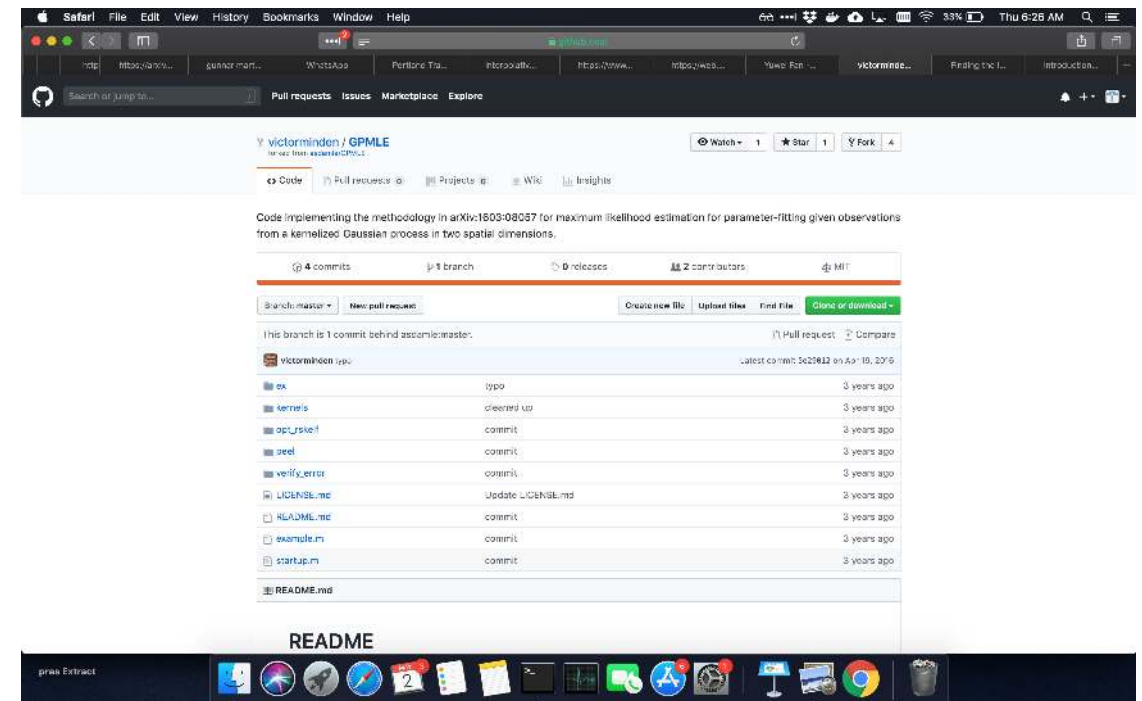


<http://dfm.io/george/current/>

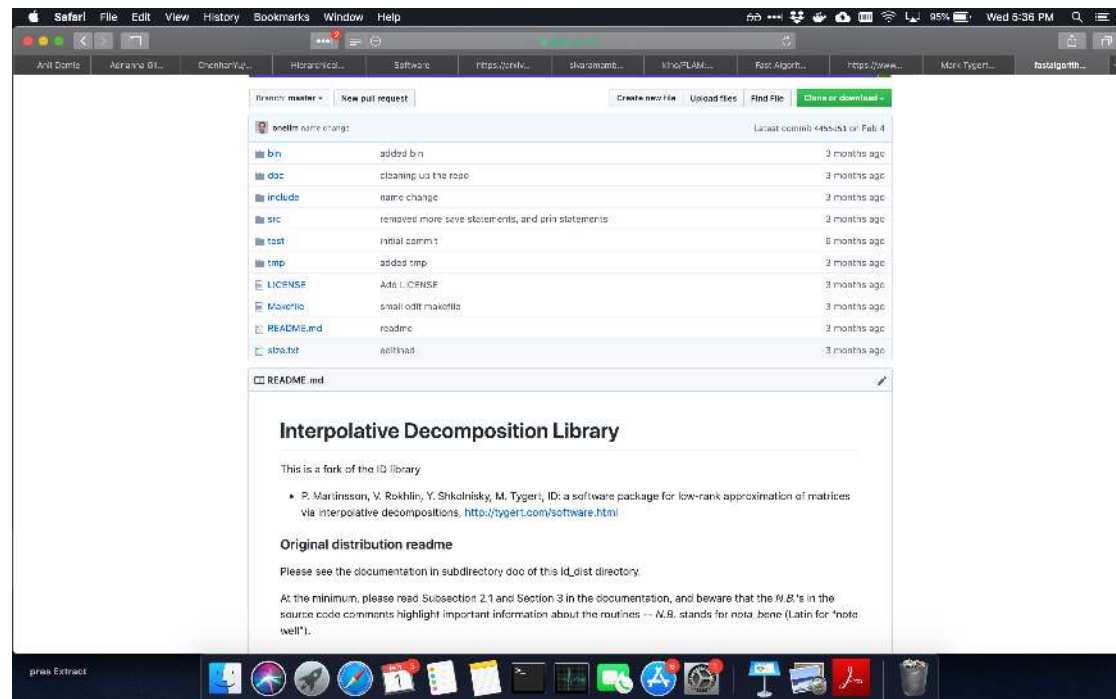
More resources



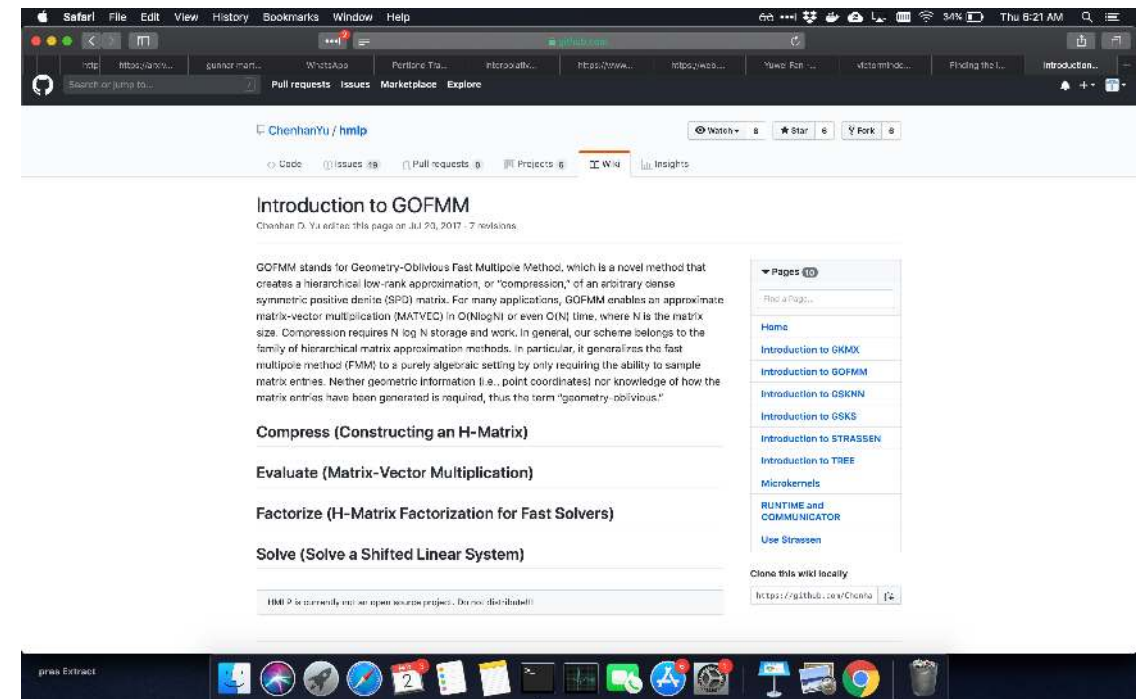
<https://github.com/danspielman/Laplacians.jl>



<https://github.com/victorminden/GPMLE>



<https://github.com/fastalgorithms/libid>



<https://github.com/ChenhanYu/hmlp/wiki/Introduction-to-GOFMM>

More resources

- Video lectures by Gunnar - https://www.youtube.com/playlist?list=PLPDZ9rcIfxyOrlpcu_D1PRcyK-o2iofwW
- Excellent review article on randomized methods for low rank approximations - Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions: <https://arxiv.org/pdf/0909.4061.pdf>
- Some of the illustrations courtesy: Sivaram Ambikasaran, Dan Foreman Mackey, David Hogg, Mike O'Neil, Per-Gunnar Martinsson, Ken Ho, Leslie Greengard, Lexing Ying, Adrianna Gillman

Even more references

- Ho, Kenneth L., and Leslie Greengard. "A fast direct solver for structured linear systems by recursive skeletonization." *SIAM Journal on Scientific Computing* 34.5 (2012): A2507-A2532.
- Yu, Chenhan D., et al. "Geometry-oblivious FMM for compressing dense SPD matrices." *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 2017.
- Ambikasaran, S., Foreman-Mackey, D., Greengard, L., Hogg, D. W., & O'Neil, M. (2016). Fast direct methods for Gaussian processes. *IEEE transactions on pattern analysis and machine intelligence*, 38(2), 252-265.
- Gillman, A., Young, P. M., & Martinsson, P. G. (2012). A direct solver with $O(N)$ complexity for integral equations on one-dimensional domains. *Frontiers of Mathematics in China*, 7(2), 217-247.
- Halko, N., Martinsson, P. G., & Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM review*, 53(2), 217-288.
- Greengard, L., & Rokhlin, V. (1991). On the numerical solution of two-point boundary value problems. *Communications on Pure and Applied Mathematics*, 44(4), 419-452.
- Fan, Y., Lin, L., Ying, L., & Zepeda-Núñez, L. (2018). A multiscale neural network based on hierarchical matrices. *arXiv preprint arXiv:1807.01883*.
- Minden, V., Damle, A., Ho, K. L., & Ying, L. (2017). Fast spatial gaussian process maximum likelihood estimation via skeletonization factorizations. *Multiscale Modeling & Simulation*, 15(4), 1584-1611.
- Martinsson, P. G., Rokhlin, V., Shkolnisky, Y., & Tygert, M. (2008). ID: A software package for low-rank approximation of matrices via interpolative decompositions, Version 0.2.
- Corona, E., Martinsson, P. G., & Zorin, D. (2015). An $O(N)$ direct solver for integral equations on the plane. *Applied and Computational Harmonic Analysis*, 38(2), 284-317.
- Gimbutas, Z., & Rokhlin, V. (2003). A generalized fast multipole method for nonoscillatory kernels. *SIAM Journal on Scientific Computing*, 24(3), 796-817.

Not an exhaustive list

Thank you!

Questions?